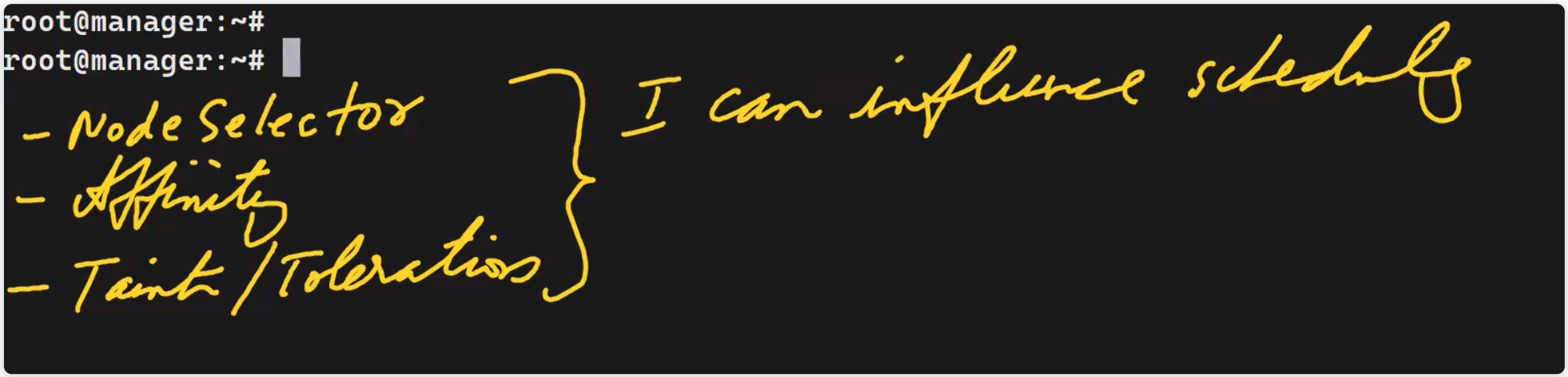
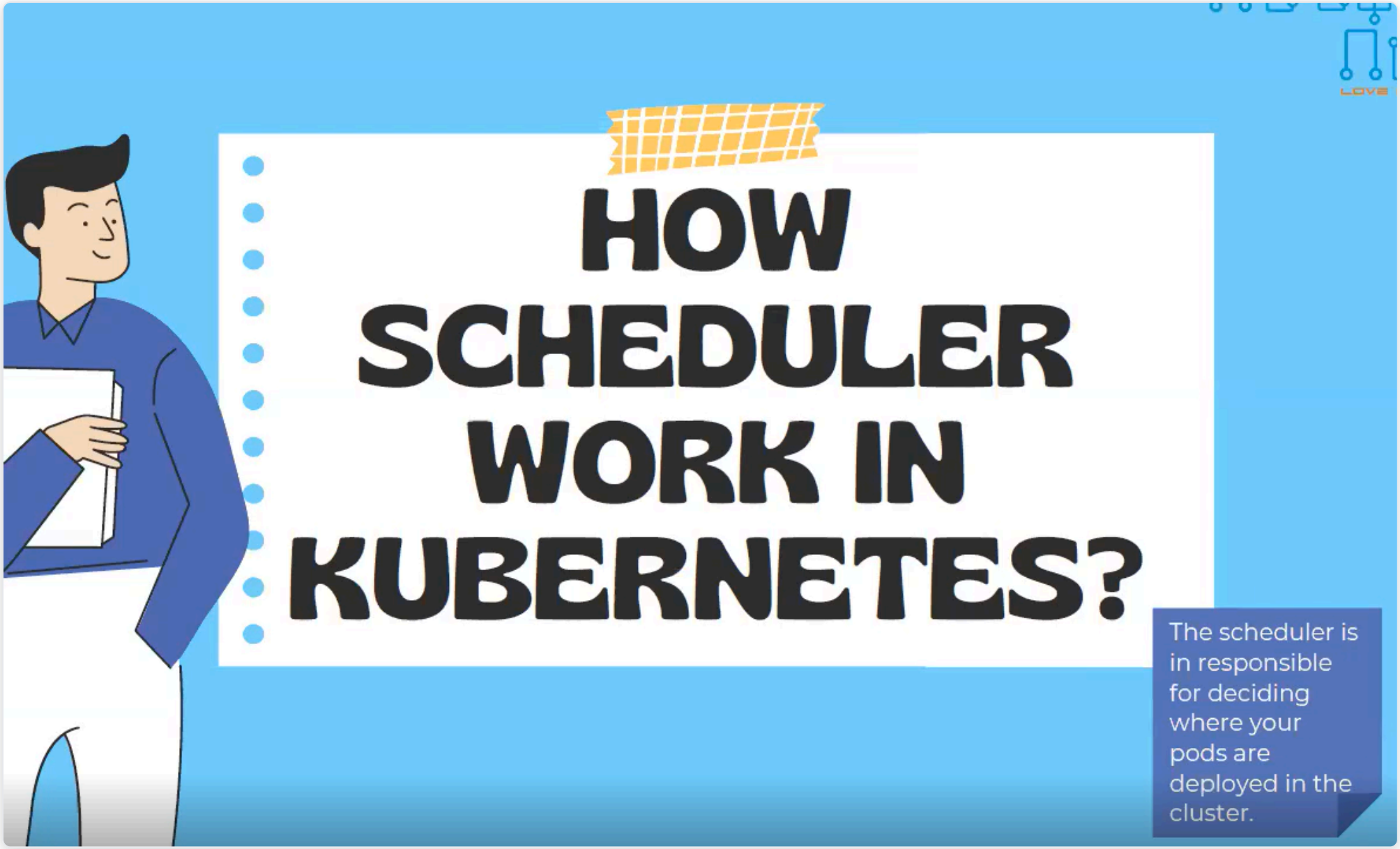
KUBERNETES CKA PREP › MY LEARNING NOTES › SESSION 06

Screenshots: Scheduler, Node Selector & Affinity

Raw screenshots captured while watching this session (scheduler internals, filter/score, node labeling, node/pod affinity), kept alongside the written notes (06-scheduler-node-selector-affinity.md).

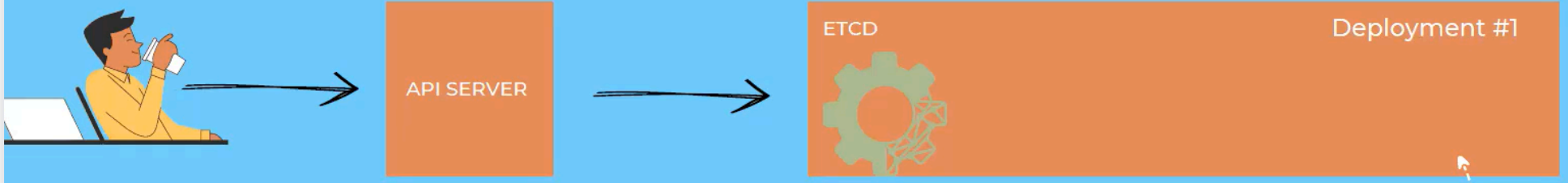
55 screenshots · captured 2026-09-13 10:30 PM - 2026-09-14 12:37 AM


root@manager:~#
root@manager:~#
- Node Selector
- Affinity
- Taints / Tolerations
I can influence scheduling

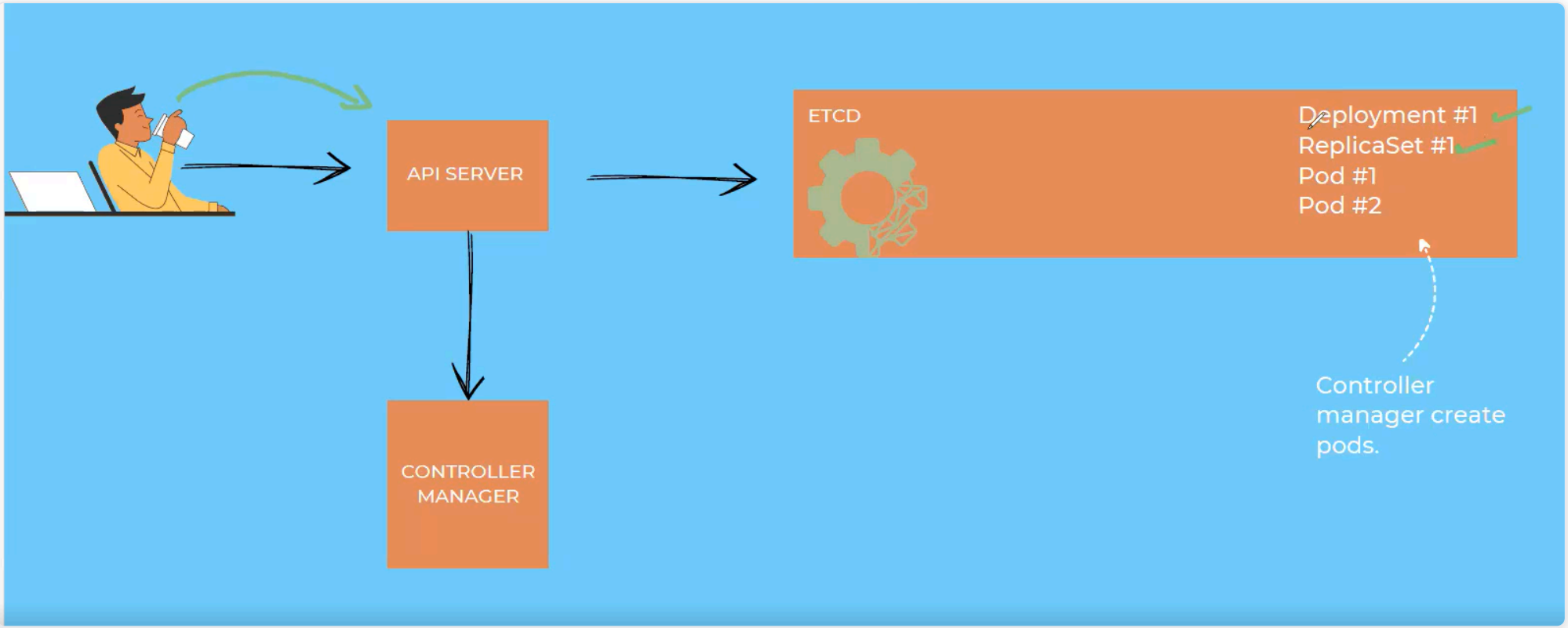

HOW SCHEDULER WORK IN KUBERNETES?
The scheduler is in responsible for deciding where your pods are deployed in the cluster.

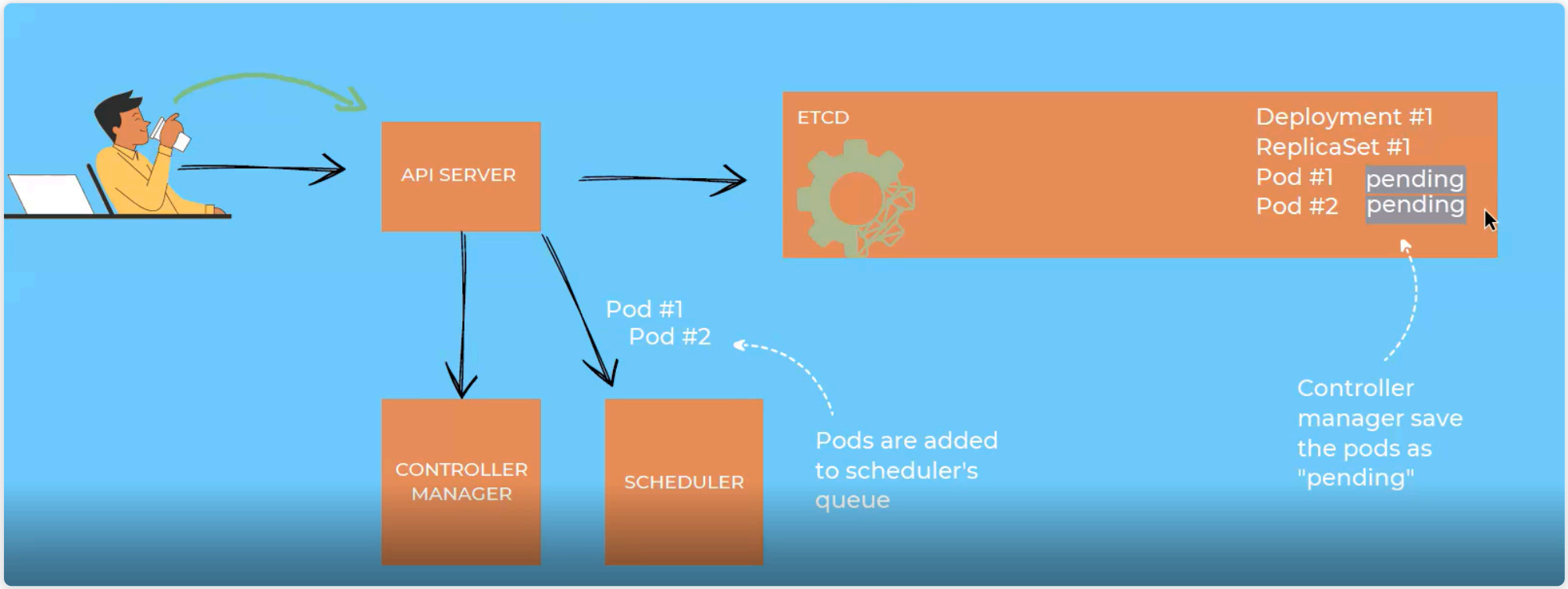
kubectl create -f deploy-abc.yaml

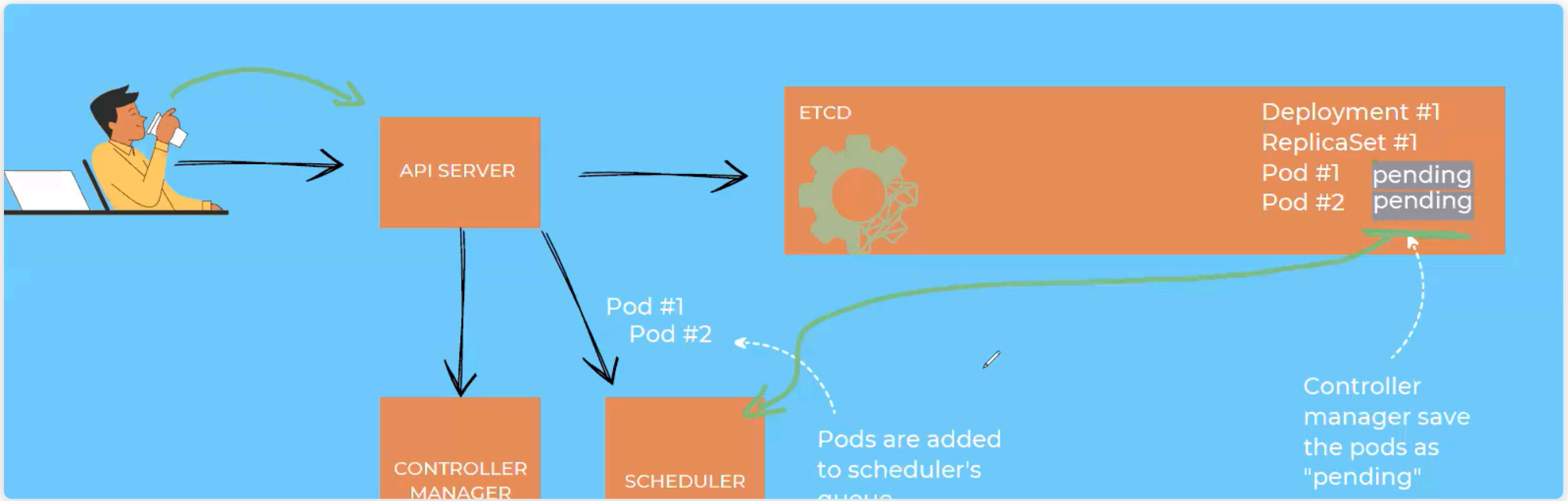
API-Server receives the request and the resrource is created in "etcd".

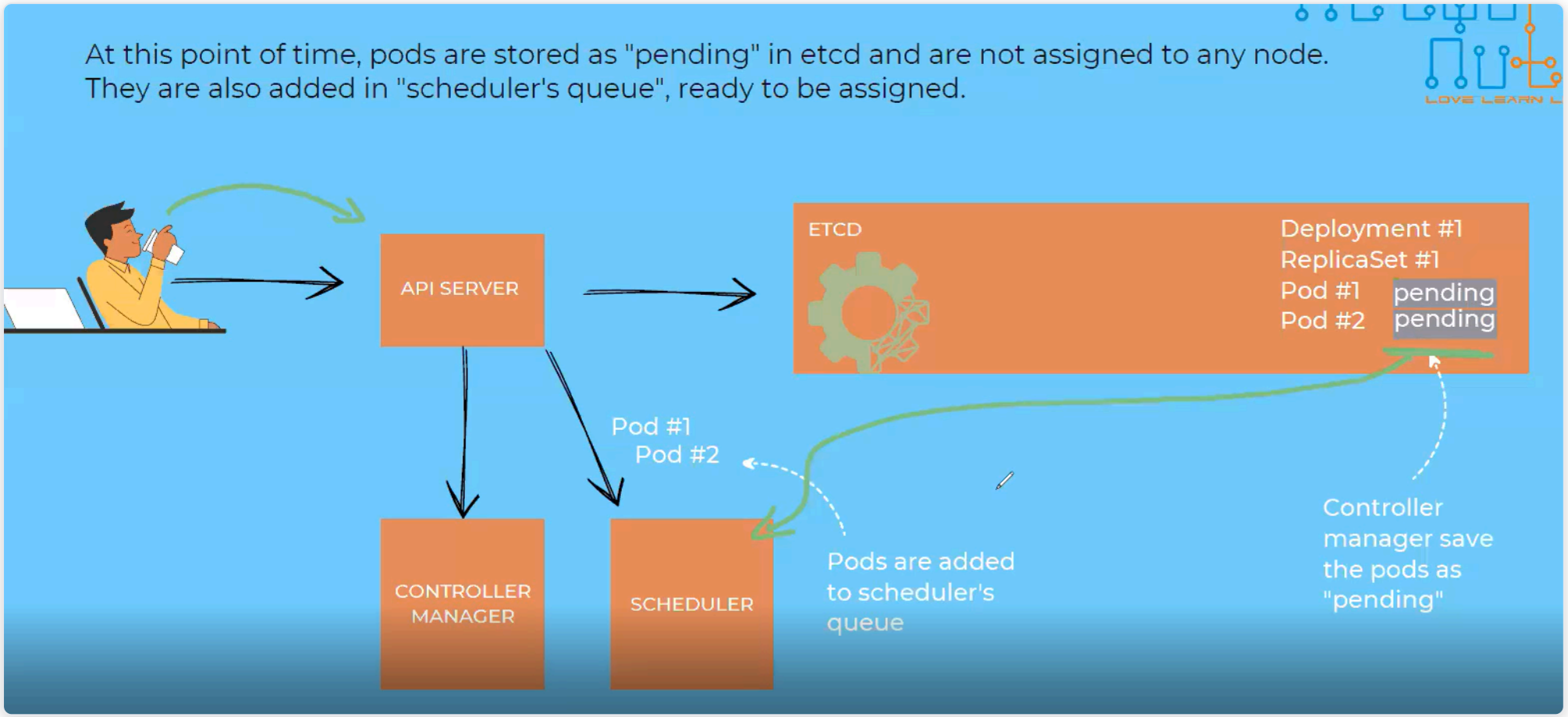


Deployment
stored in etcd


API SERVER
CONTROLLER MANAGER
ETCD
Deployment #1
ReplicaSet #1
Pod #1
Pod #2
Controller manager create pods.

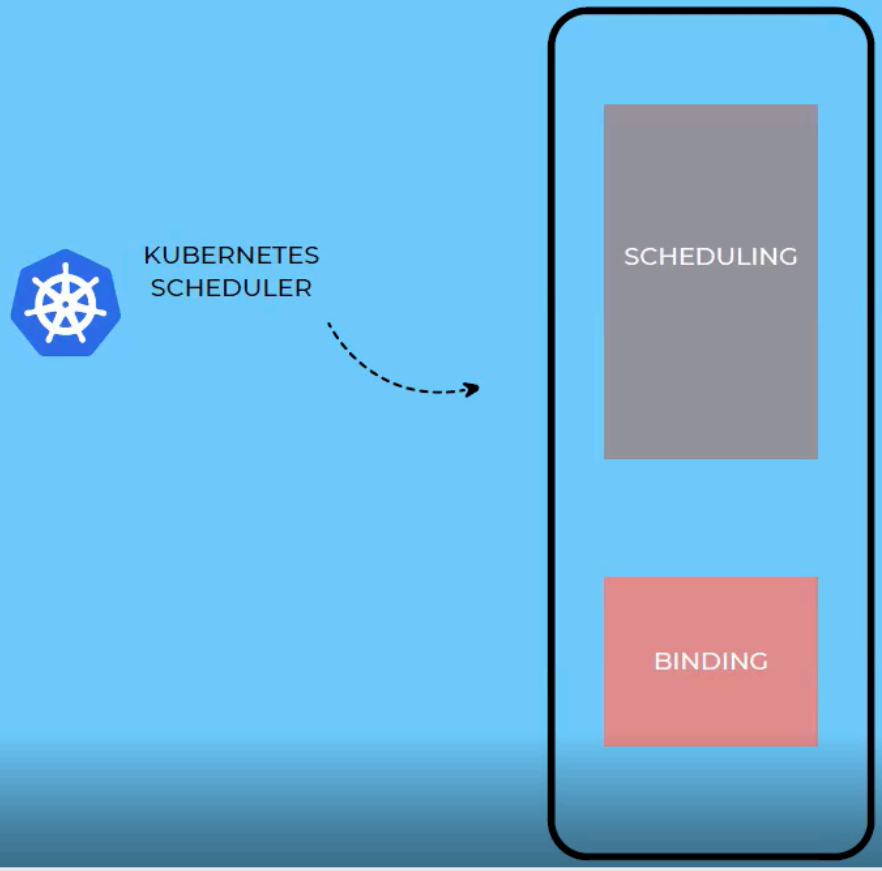

API SERVER
ETCD
Deployment #1
ReplicaSet #1
Pod #1 pending
Pod #2 pending
Pod #1
Pod #2
CONTROLLER MANAGER
SCHEDULER
Pods are added to scheduler's queue
Controller manager save the pods as "pending"


API SERVER
ETCD
Deployment #1
ReplicaSet #1
Pod #1 pending
Pod #2 pending
Pod #1
Pod #2
CONTROLLER MANAGER
SCHEDULER
Pods are added to scheduler's
Controller manager save the pods as "pending"


At this point of time, pods are stored as "pending" in etcd and are not assigned to any node.
They are also added in "scheduler's queue", ready to be assigned.
API SERVER
ETCD
Deployment #1
ReplicaSet #1
Pod #1 pending
Pod #2 pending
Pod #1
Pod #2
CONTROLLER MANAGER
SCHEDULER
Pods are added to scheduler's queue
Controller manager save the pods as "pending"

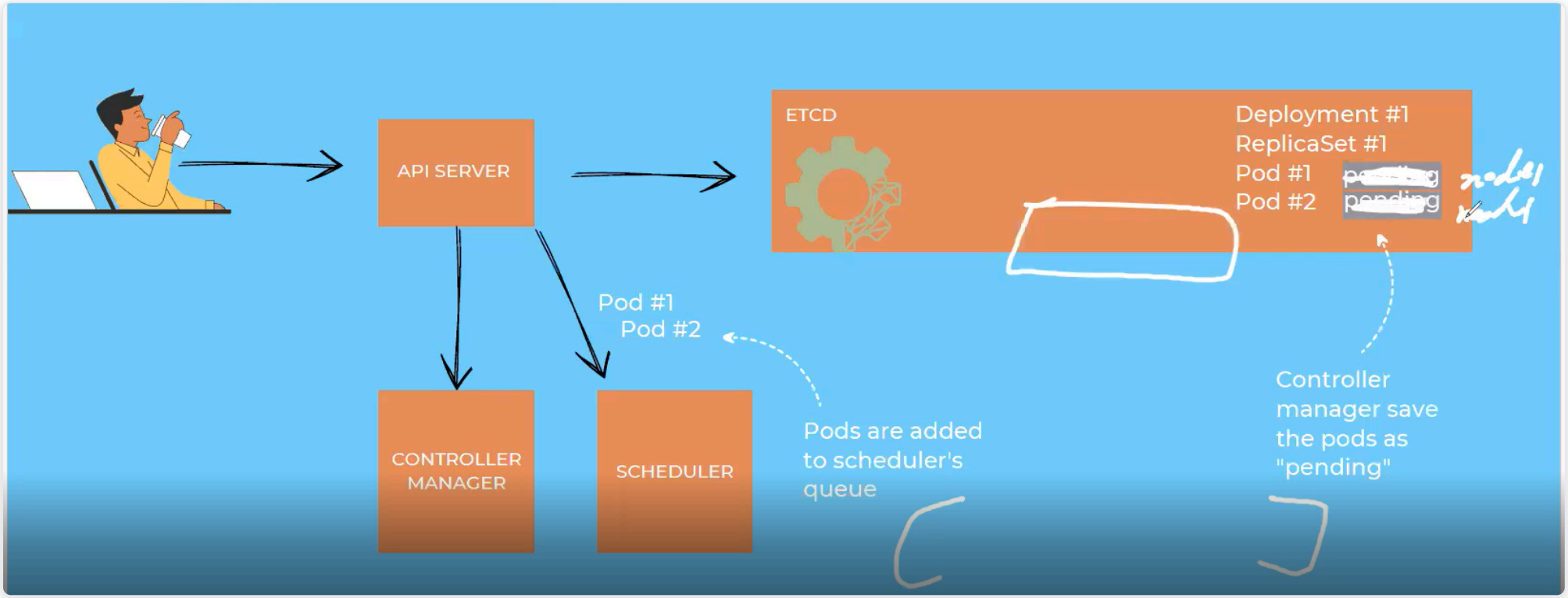
Scheduler process "Pod #1" through two steps:

1. Scheduling Phase (what node should I choose?)
2. Binding Phase (write to database i.e. etcd that this pod belong to that node)



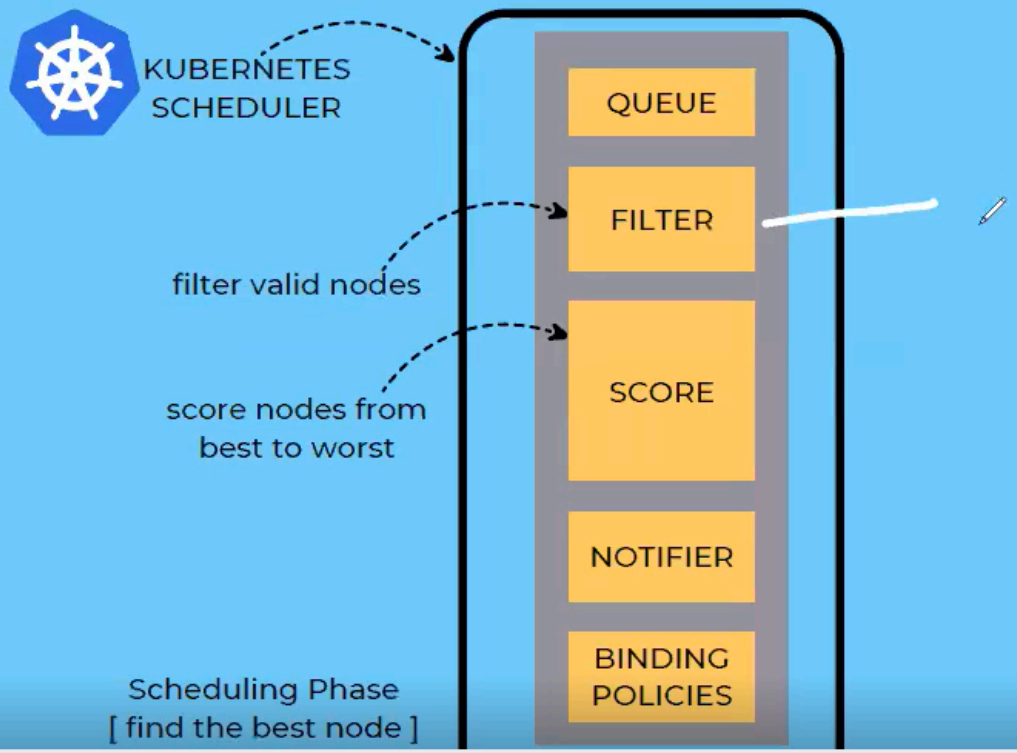
Scheduling Phase
[find the best node]

Binding Phase
[bind the pod to node & update etcd]


API SERVER
ETCD
Deployment #1
ReplicaSet #1
Pod #1
Pod #2
Pod #1
Pod #2
CONTROLLER MANAGER
SCHEDULER
Pods are added to scheduler's queue
Controller manager save the pods as "pending"

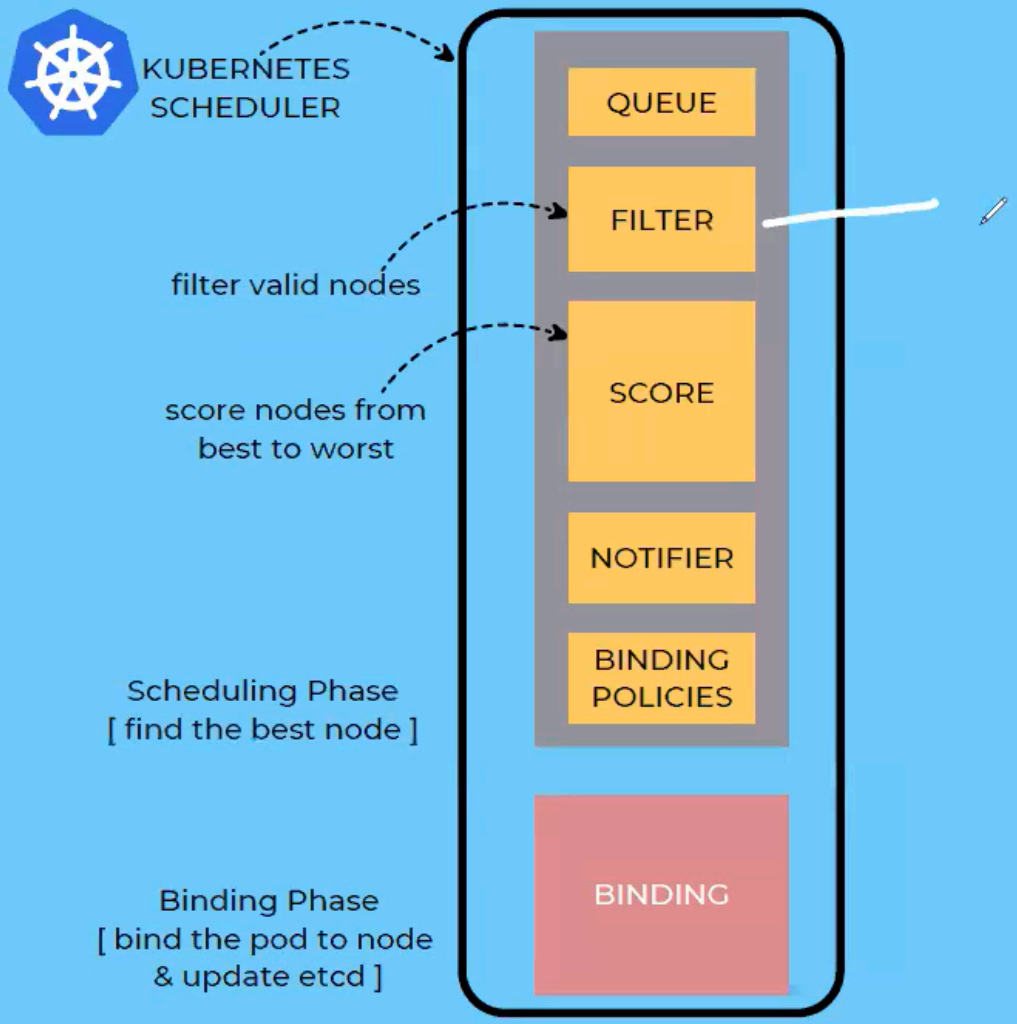
Now scheduler phase is divided into two parts:

1. Filter relevant nodes (use functions called predicates)
2. Rank remaining nodes (use functions are priorities)



Now scheduler phase is divided into two parts:

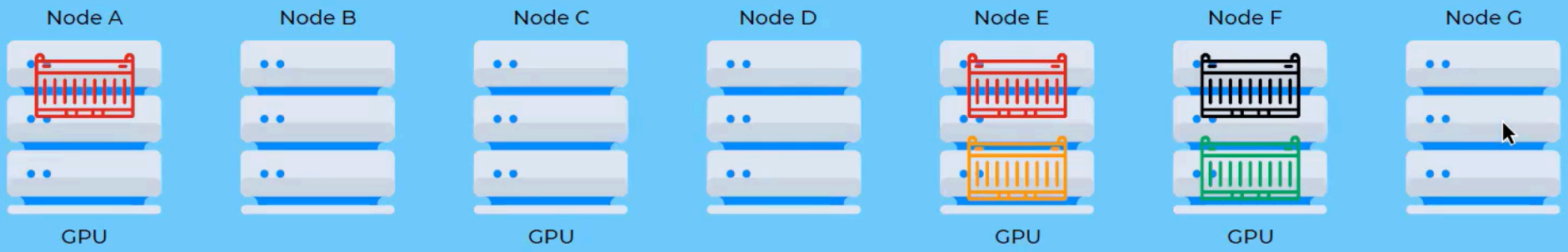
1. Filter relevant nodes (use functions called predicates)
2. Rank remaining nodes (use functions are priorities)



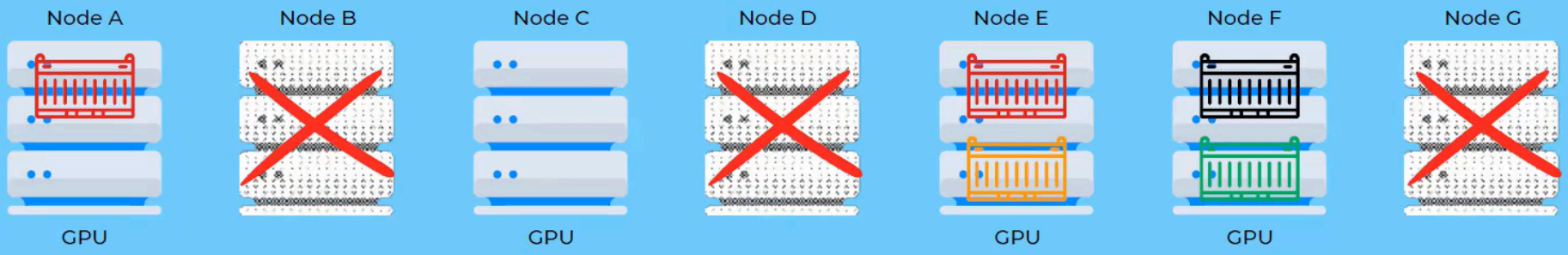
Lets take a example of a cluster.

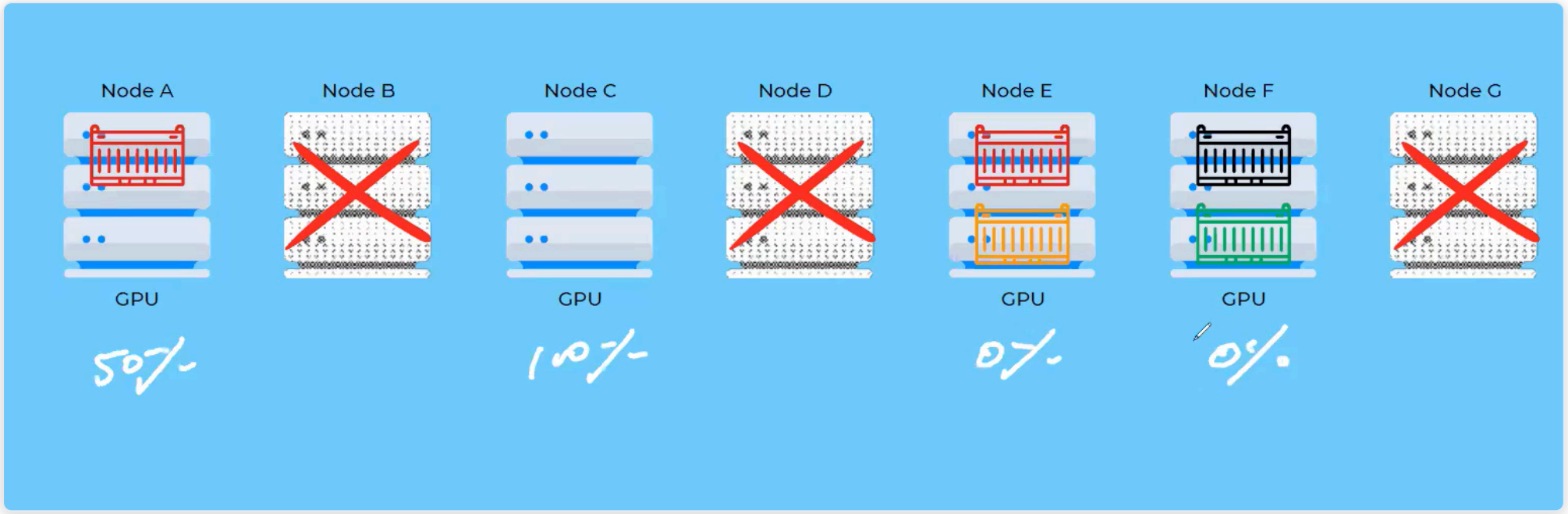
Here, we want to deploy a pod that needs GPU (or any other specific requirement like SSD storage)
We submit the request to cluster and its added to scheduler's queue.

Here, few nodes are running on full capacity.

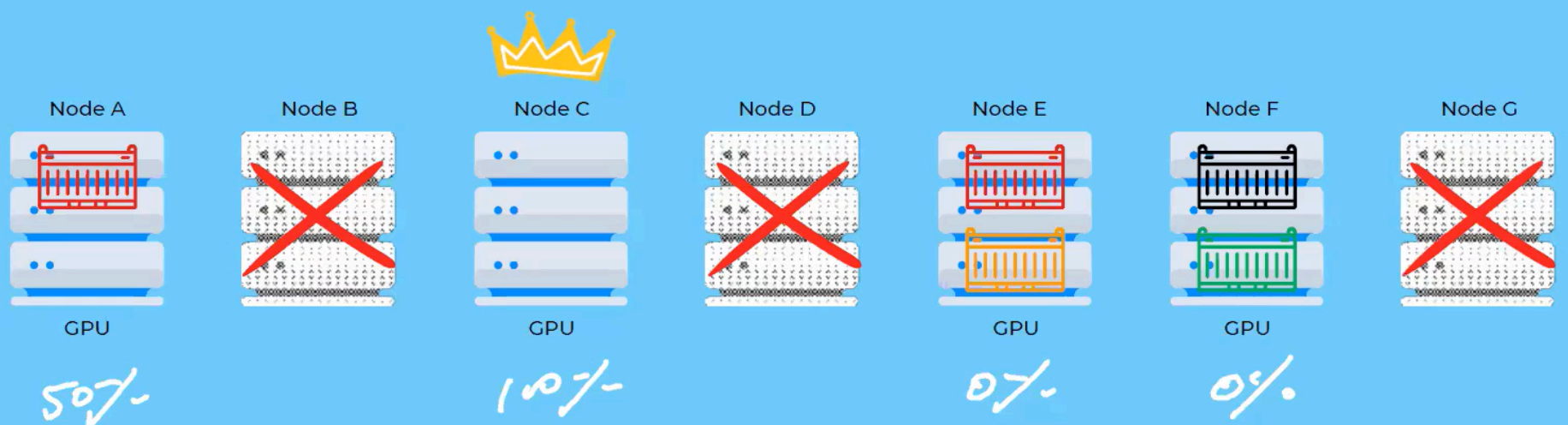


#1 - Scheduler will discard all the nodes that doesn't have GPU [filter phase]



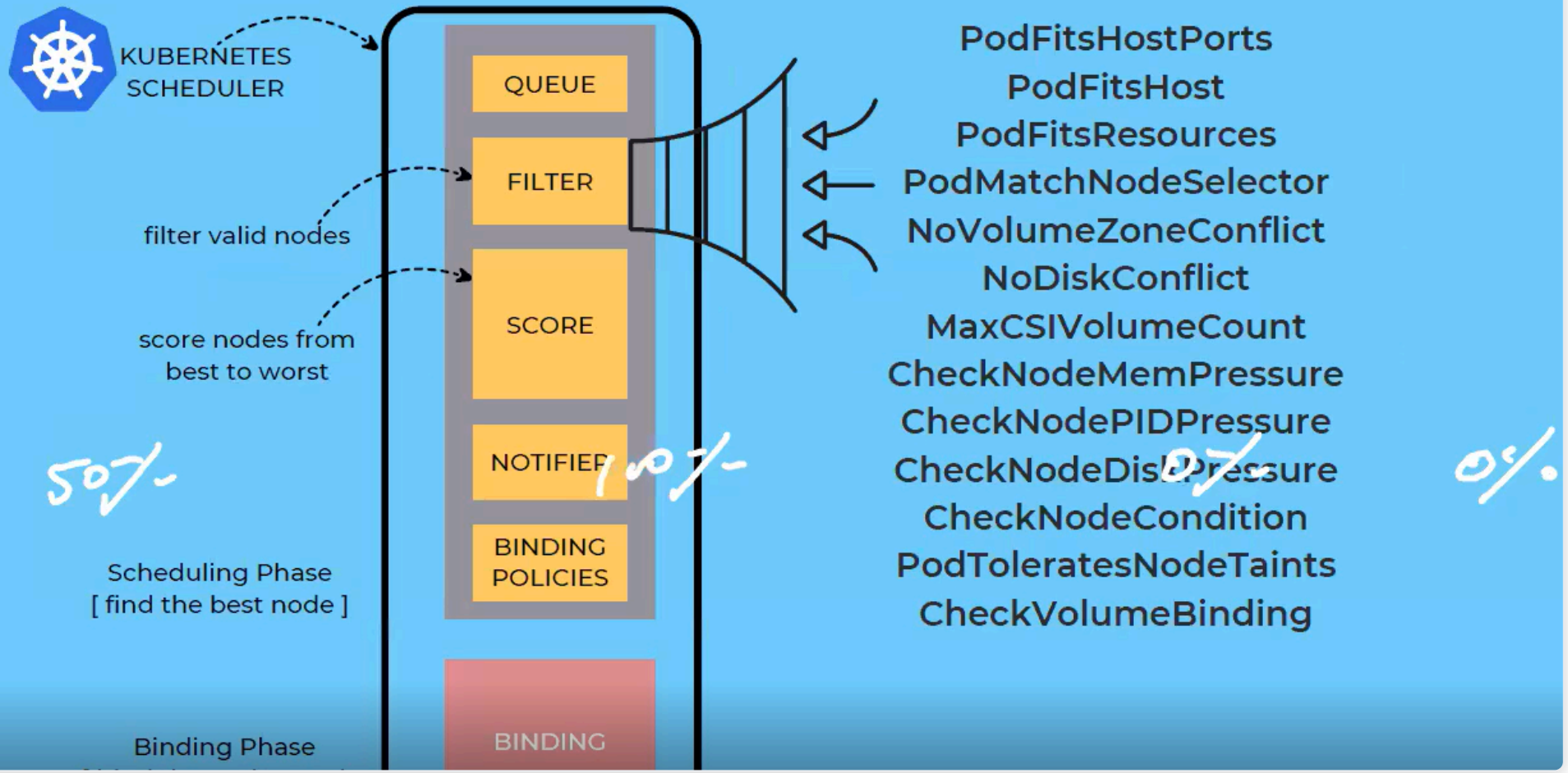

Node A
Node B
Node C
Node D
Node E
Node F
Node G
GPU
GPU
GPU
GPU
50%
100%
0%
0%

#2 - Now the scheduler score the remaining nodes (A / C / E / F). Fully utilized nodes have lower score, so empty node is selected (Node C)



Lets talk about filters. Default filter phase has 13 predicates. Some common are:

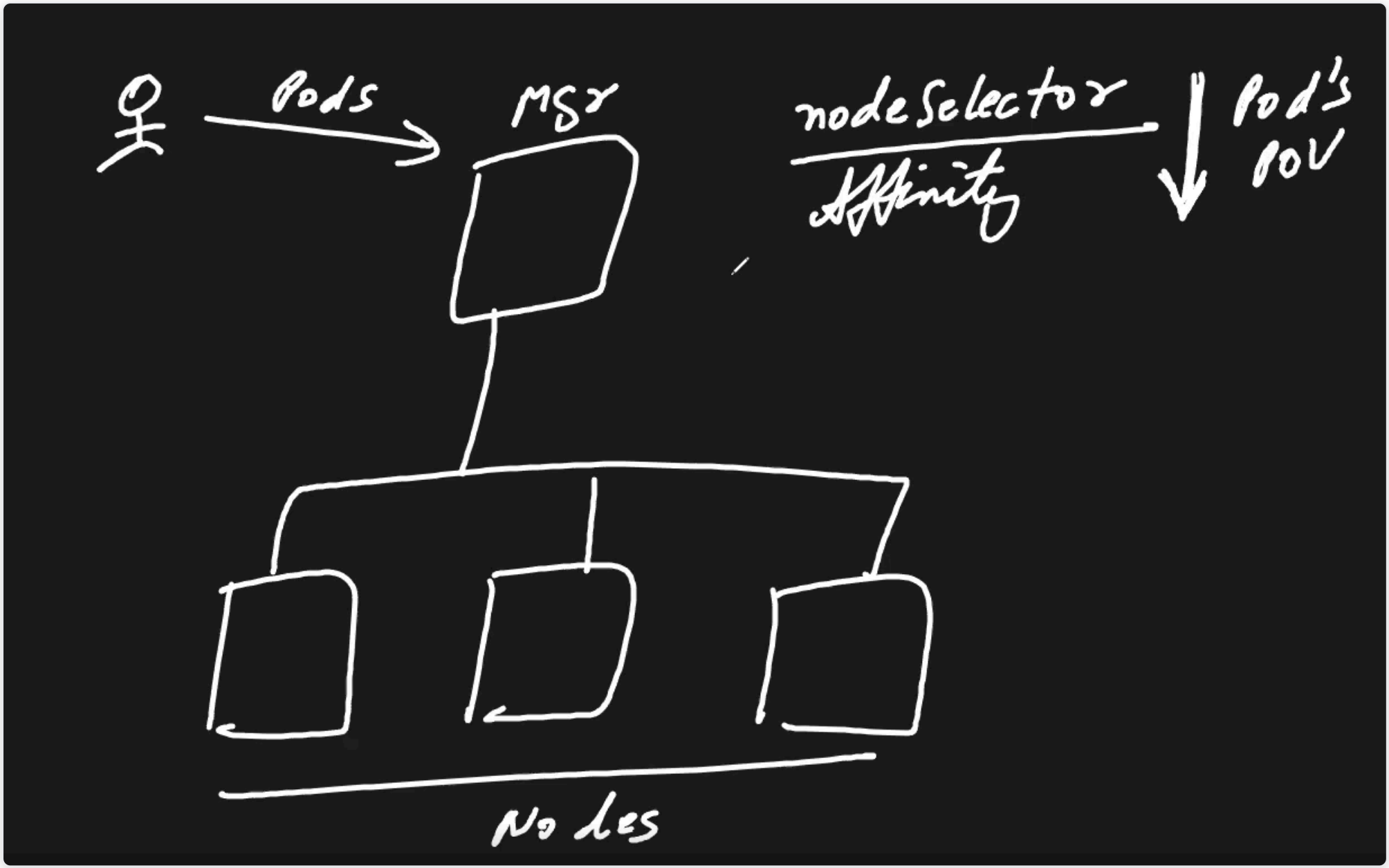
- NodeUnScheduleable - no pod land on this node
- VolumeBinding - if node can bind requested volume

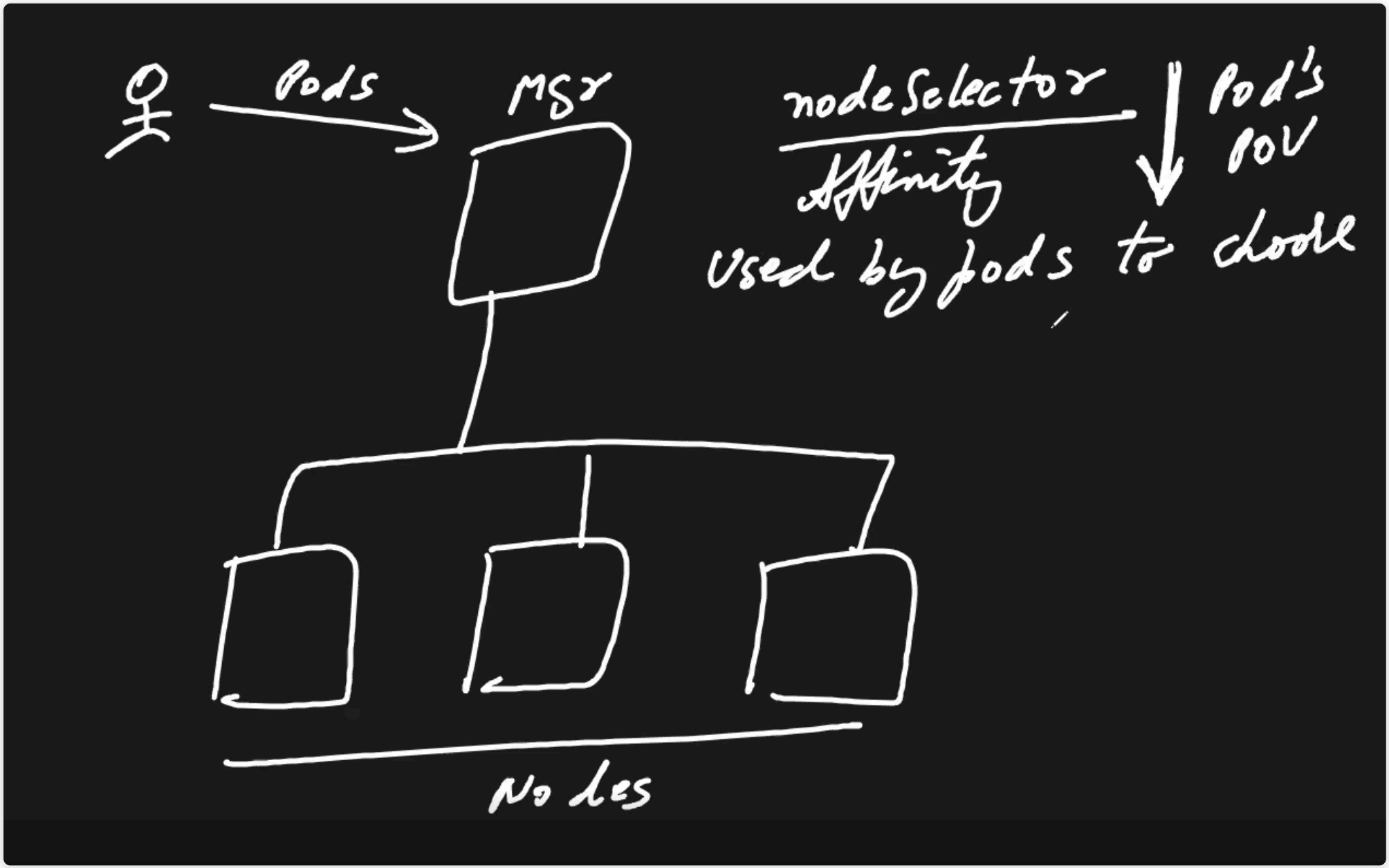


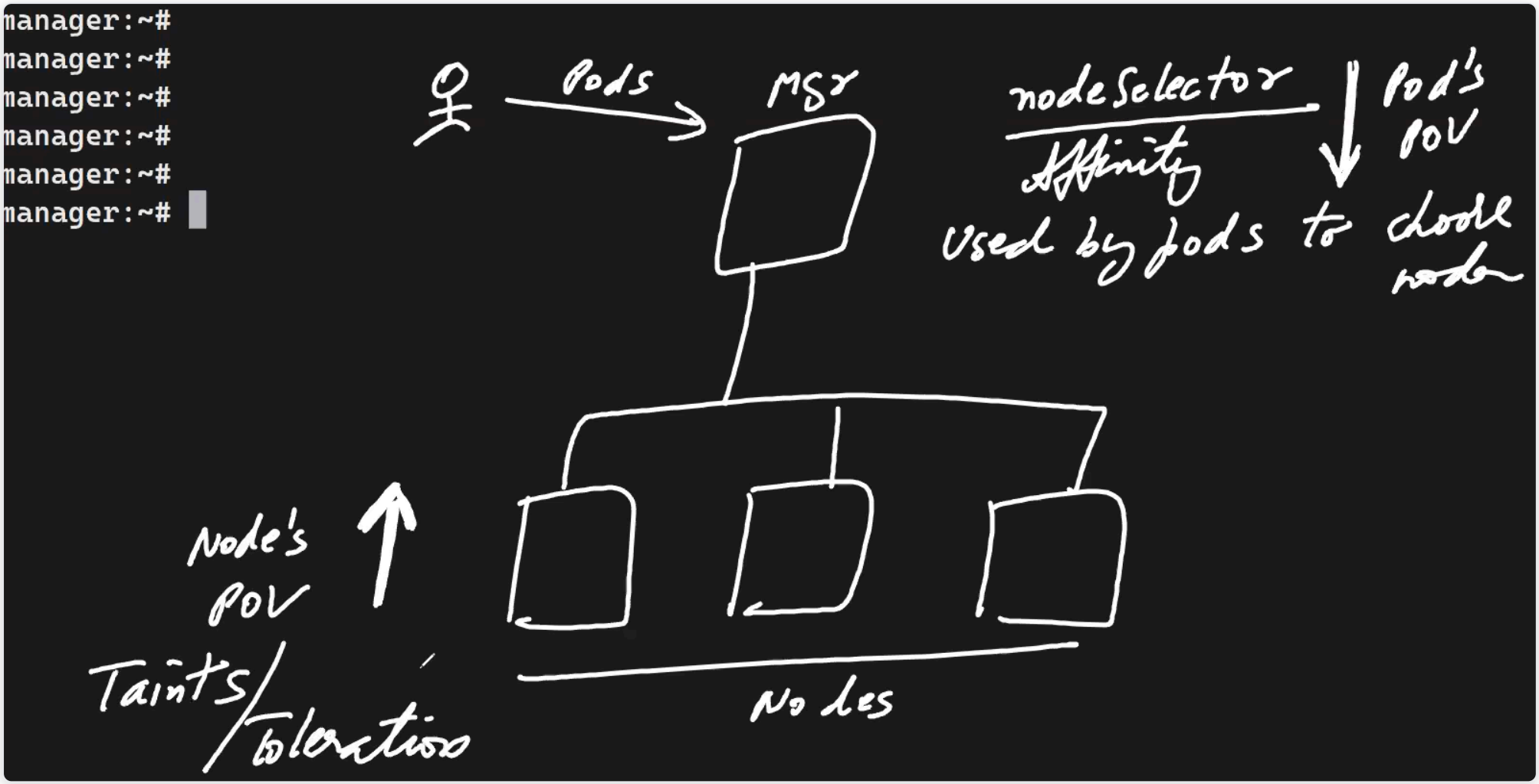
YES, we can using:

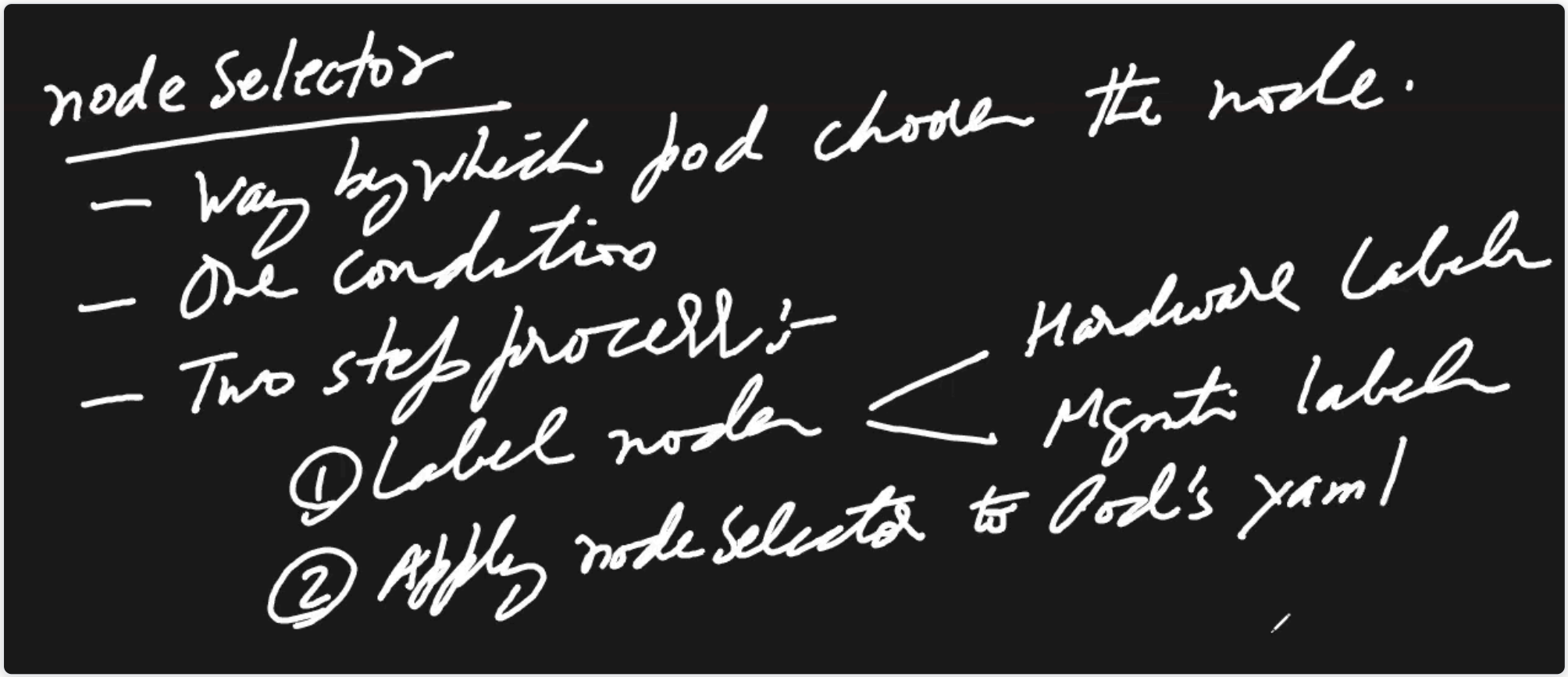
- nodeSelector
- NodeAffinity
- Pod Affinity / Anti Affinity
- Taints and Toleration
- Topology Constraints
- Scheduler Profiles

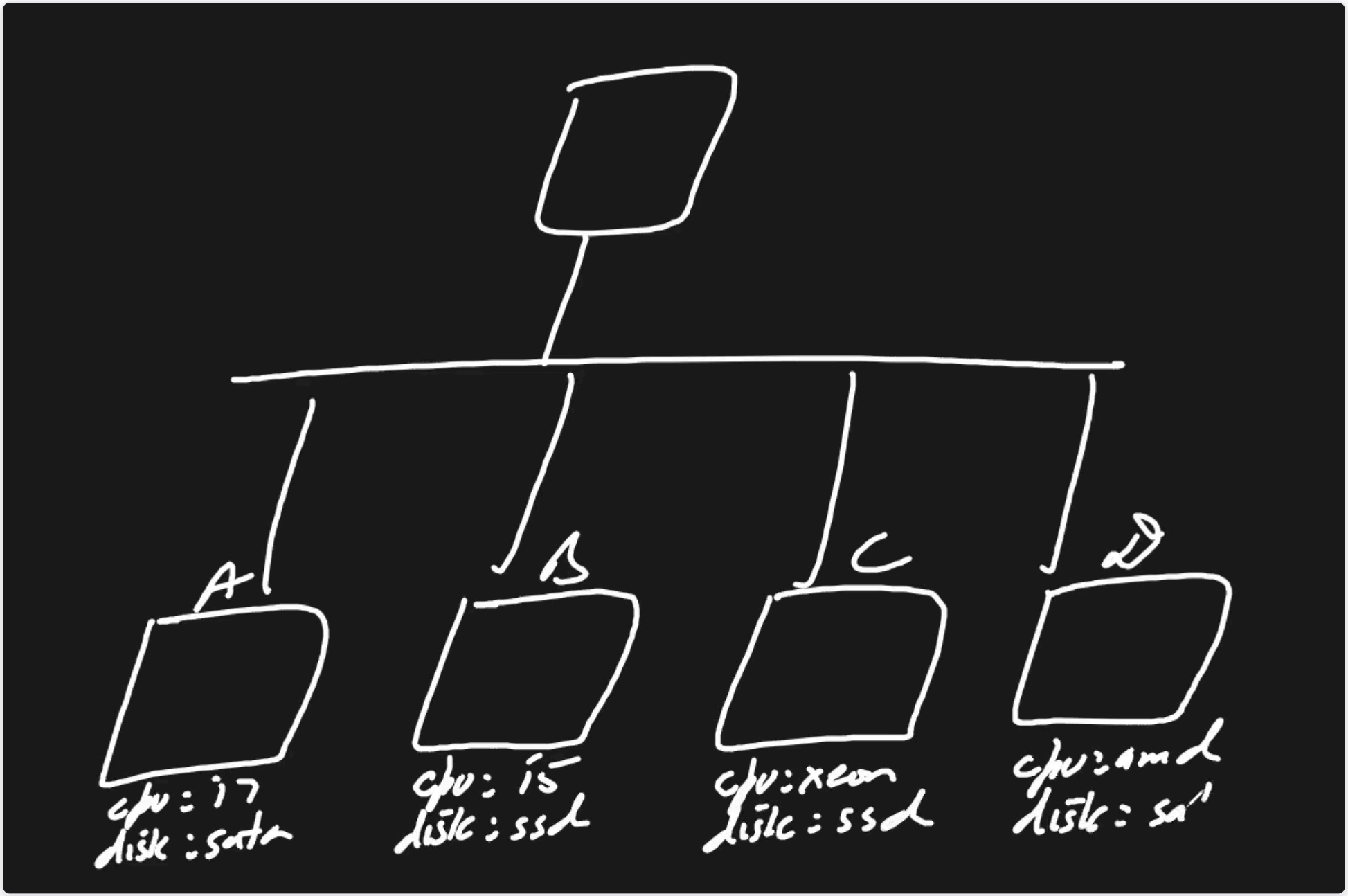
We can also use "scheduler policies" to customize how the scheduler uses filters and predicates to assign nodes to pods. Available after version 1.25

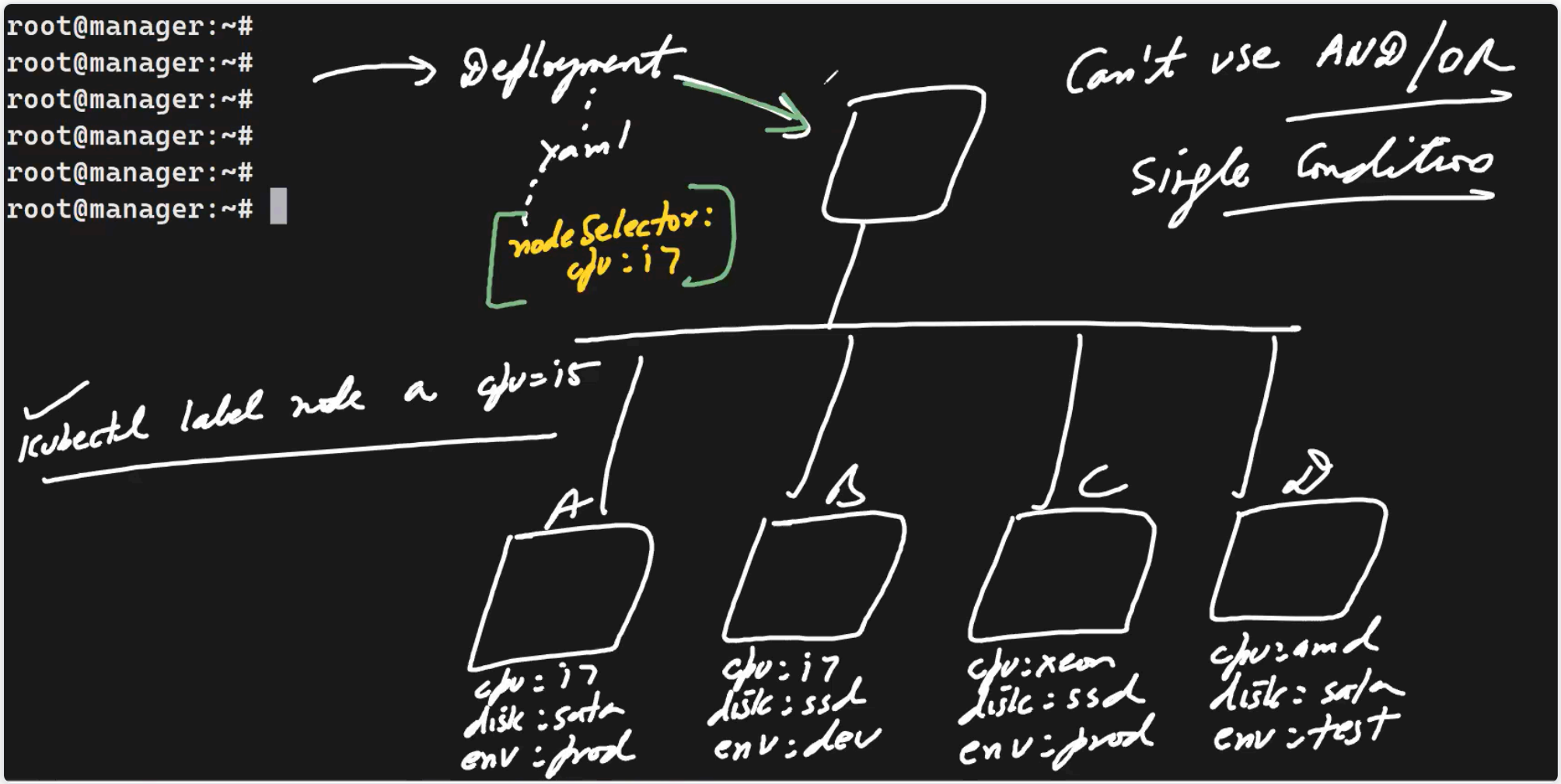

Pods
MSr
nodeSelector
Affinity
Pod's
POV
Nodes


Pods
MSr
nodeSelector
Affinity
Pod's POV
Used by pods to choose
Nodes


manager:~#
manager:~#
manager:~#
manager:~#
manager:~#
manager:~#
Pods
MSr
nodeSelector
Affinity
Pod's POV
Used by pods to choose node
Node's POV
Taints/Tolerations
Nodes

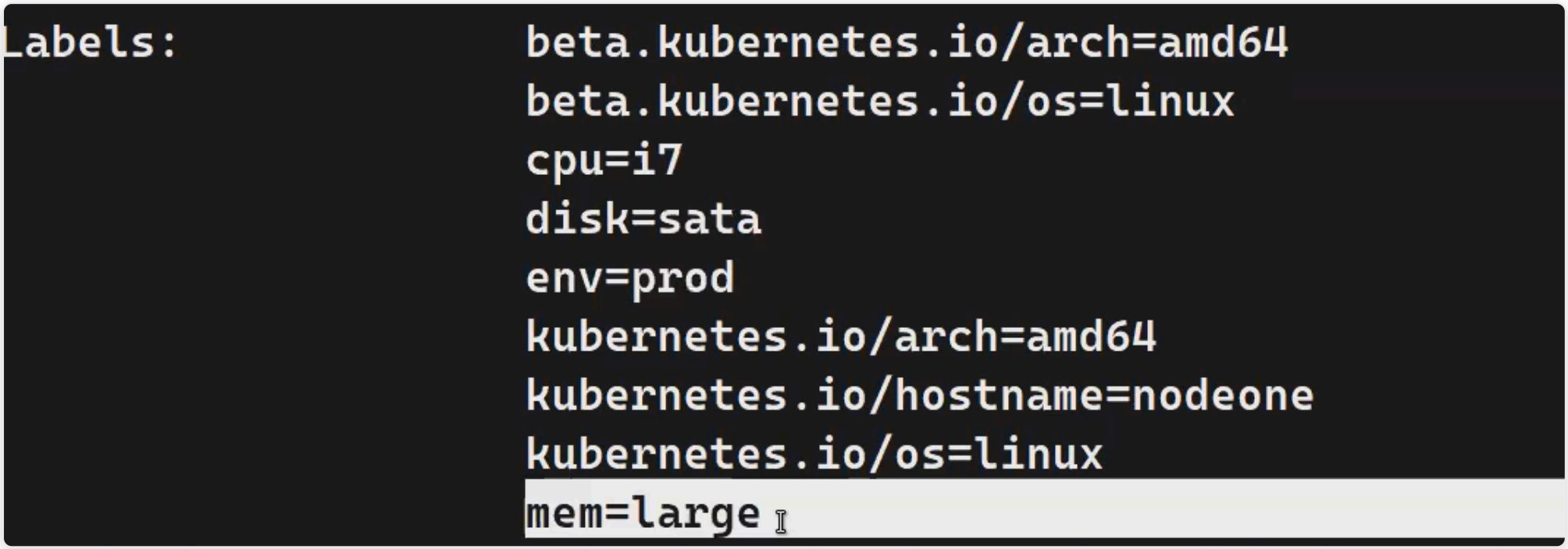

node Selector
– Way by which pod choose the node.
– One condition
– Two step process :-
① Label nodes
② Apply nodeSelector
Hardware labels
Mgmt labels
to Pod's yaml


A
B
C
D
cpu : i7
disk : sata
cpu : i5
disk : ssd
cpu : xeon
disk : ssd
cpu : amd
disk : sa

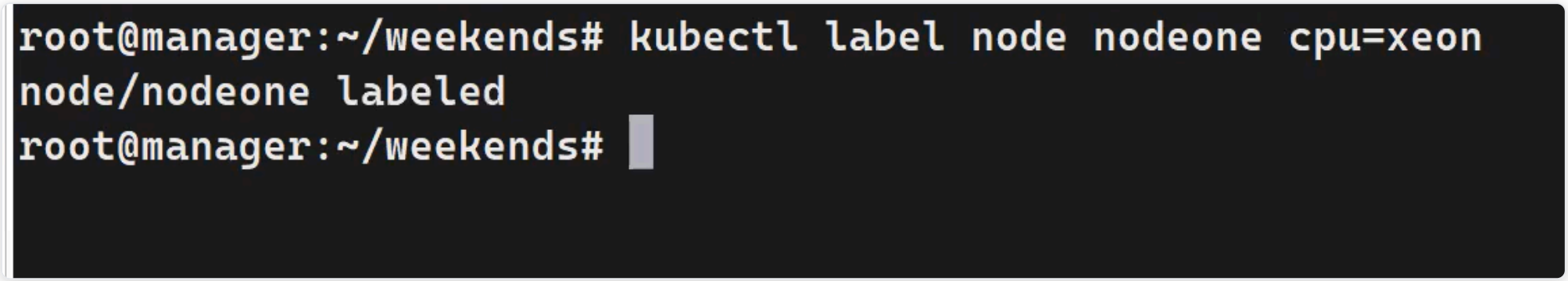

root@manager:~#
root@manager:~#
root@manager:~#
root@manager:~#
root@manager:~#
root@manager:~#
Deployment
.yaml
nodeSelector:
cpu: i7
Can't use AND/OR
Single Condition
kubectl label node a cpu=i5
A
B
C
D
cpu: i7
disk: sata
env: prod
cpu: i7
disk: ssd
env: dev
cpu: xeon
disk: ssd
env: prod
cpu: amd
disk: sata
env: test

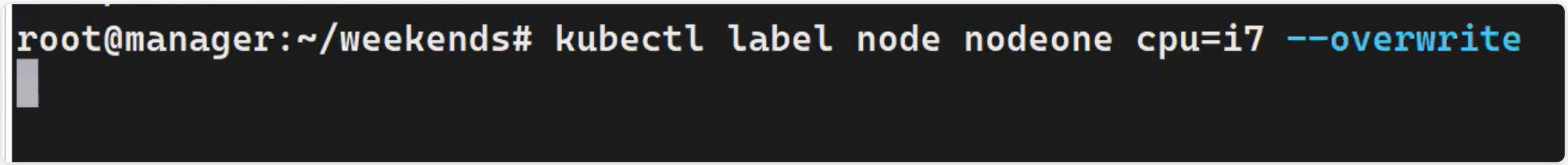
```
root@manager:~# kubectl describe node nodeone
Name:               nodeone
Roles:              <none>
Labels:             beta.kubernetes.io/arch=amd64
                    beta.kubernetes.io/os=linux
                    kubernetes.io/arch=amd64
                    kubernetes.io/hostname=nodeone
                    kubernetes.io/os=linux
```

```
root@manager:~/weekends# kubectl label node nodeone cpu=i7
node/nodeone labeled
root@manager:~/weekends# kubectl label node nodeone mem=large
node/nodeone labeled
root@manager:~/weekends# kubectl label node nodeone disk=sata
node/nodeone labeled
root@manager:~/weekends# kubectl label node nodeone env=prod
```


Labels:
beta.kubernetes.io/arch=amd64
beta.kubernetes.io/os=linux
cpu=i7
disk=sata
env=prod
kubernetes.io/arch=amd64
kubernetes.io/hostname=nodeone
kubernetes.io/os=linux
mem=large

```
root@manager:~/weekends# kubectl label node nodeone cpu-
node/nodeone unlabeled
root@manager:~/weekends#
```


root@manager:~/weekends# kubectl label node nodeone cpu=xeon
node/nodeone labeled
root@manager:~/weekends#


root@manager:~/weekends# kubectl label node nodeone cpu=i7 --overwrite

```
root@manager:~/weekends# kubectl get nodes -L cpu -L disk -L mem -L env
NAME      STATUS   ROLES           AGE    VERSION   CPU   DISK   MEM     ENV
manager   Ready    control-plane   2d8h   v1.27.1
nodeone   Ready    <none>          160m   v1.27.1   i7    sata   large   prod
nodetwo   Ready    <none>          160m   v1.27.1   i5    ssd    small   dev
```

```yaml
metadata:
  name: webapp
spec:
  selector:
    matchLabels:
      run: nn-web
  replicas: 2
  template:
    metadata:
      labels:
        run: nn-web
    spec:
      containers:
      - name: nn-webserver
        image: lovelearnlinux/webserver:v1
        ports:
        - containerPort: 80
      nodeSelector:
        disk: ssd
```

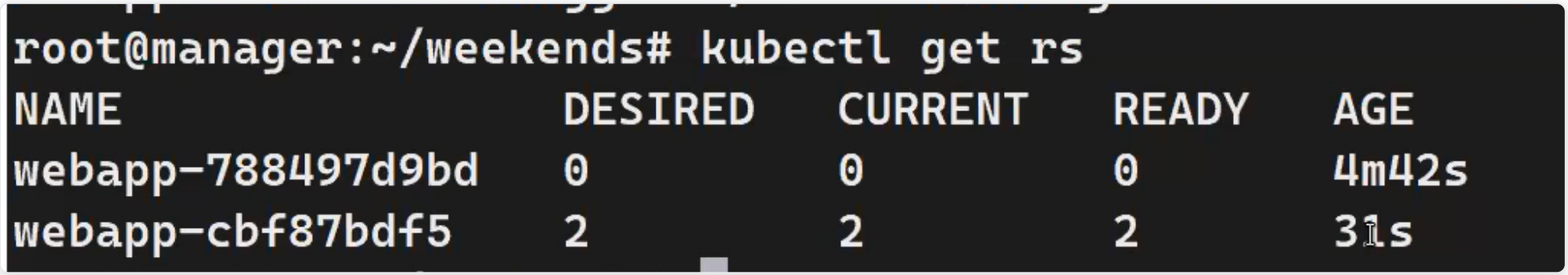
```yaml
template:
  metadata:
    labels:
      run: nn-web
  spec:
    containers:
    - name: nn-webserver
      image: lovelearnlinux/webserver:v1
      ports:
      - containerPort: 80
      resources:
        limits:
          cpu: 100m
          memory: 128Mi
        requests:
          cpu: 80m
          memory: 100Mi
    nodeSelector:
      disk: ssd
```

```
root@manager:~/weekends# kubectl create -f mydep.yaml
deployment.apps/webapp created
root@manager:~/weekends# kubectl get deploy
NAME     READY   UP-TO-DATE   AVAILABLE   AGE
webapp   0/2     2            0           4s
root@manager:~/weekends# kubectl get rs
NAME                DESIRED   CURRENT   READY   AGE
webapp-cbf87bdf5    2         2         0       7s
root@manager:~/weekends# kubectl get pods
NAME                      READY   STATUS              RESTARTS   AGE
webapp-cbf87bdf5-rpj9n    0/1     ContainerCreating   0          9s
webapp-cbf87bdf5-vh7lw    0/1     ContainerCreating   0          9s
```

```
root@manager:~/weekends# kubectl get pods -o wide
NAME                      READY   STATUS    RESTARTS   AGE   IP              NODE      NOMINATED
NODE   READINESS GATES
webapp-cbf87bdf5-rpj9n    1/1     Running   0          12s   172.16.3.131    nodetwo   <none>
       <none>
webapp-cbf87bdf5-vh7lw    1/1     Running   0          12s   172.16.3.132    nodetwo   <none>
       <none>
```

```
    Type:                    Projected (a volume that contains injected data from multiple s
ces)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    ConfigMapOptional:       <nil>
    DownwardAPI:             true
QoS Class:                   Burstable
Node-Selectors:              disk=ssd
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type     Reason     Age   From               Message
  ----     ------     ----  ----               -------
```

```
root@manager:~/weekends# kubectl get deploy
NAME     READY   UP-TO-DATE   AVAILABLE   AGE
webapp   0/2     2            0           2m43s
root@manager:~/weekends# kubectl get rs
NAME                DESIRED   CURRENT   READY   AGE
webapp-788497d9bd   2         2         0       2m45s
root@manager:~/weekends# kubectl get pods
NAME                      READY   STATUS    RESTARTS   AGE
webapp-788497d9bd-8cr5c   0/1     Pending   0          2m48s
webapp-788497d9bd-sqg8q   0/1     Pending   0          2m48s
root@manager:~/weekends#
root@manager:~/weekends# kubectl edit deployment webapp
deployment.apps/webapp edited
root@manager:~/weekends#
root@manager:~/weekends# kubectl get rs
NAME                DESIRED   CURRENT   READY   AGE
webapp-788497d9bd   2         2         0       4m15s
webapp-cbf87bdf5    1         1         0       4s
```

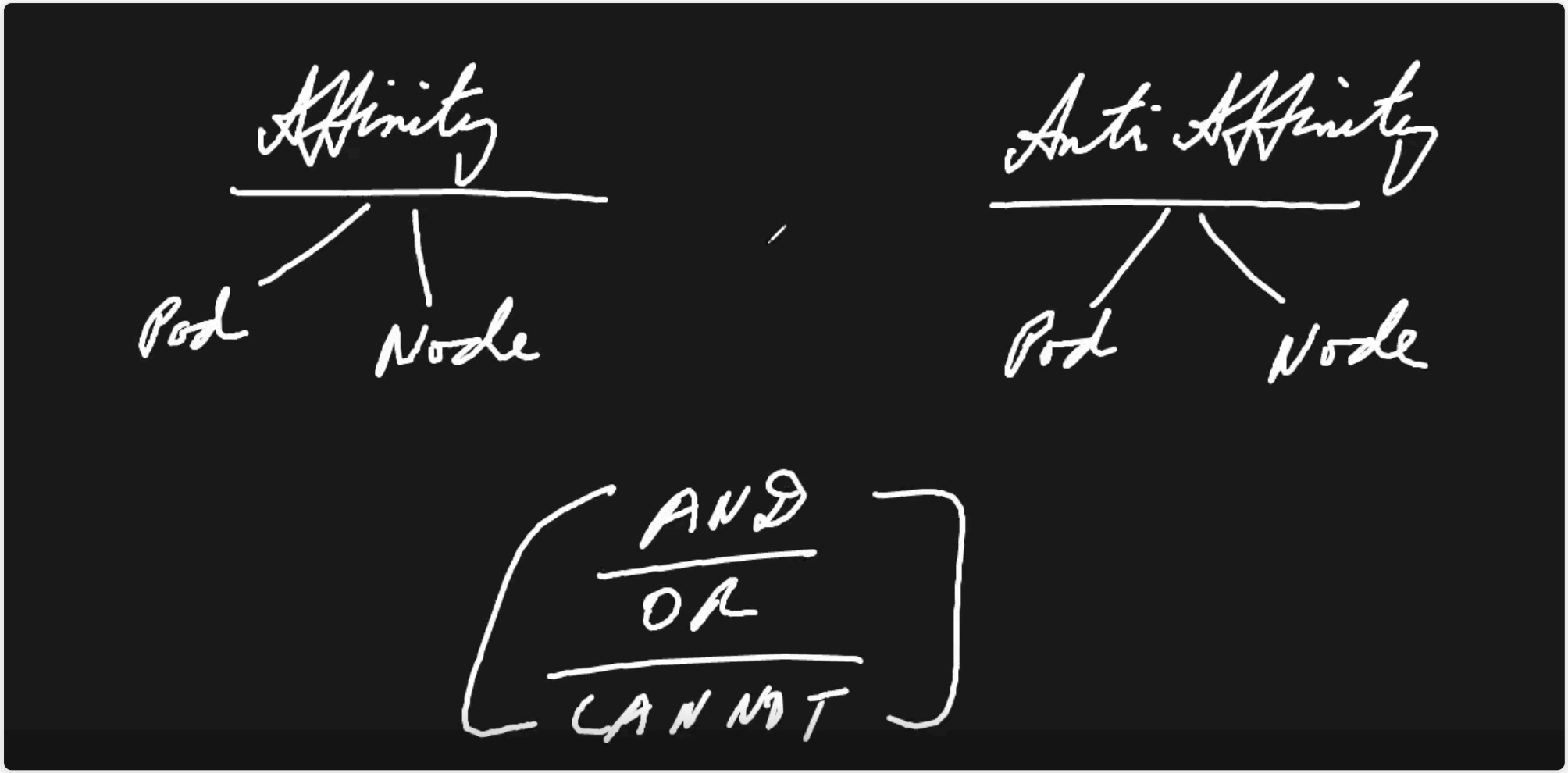

root@manager:~/weekends# kubectl get rs
NAME DESIRED CURRENT READY AGE
webapp-788497d9bd 0 0 0 4m42s
webapp-cbf87bdf5 2 2 2 31s

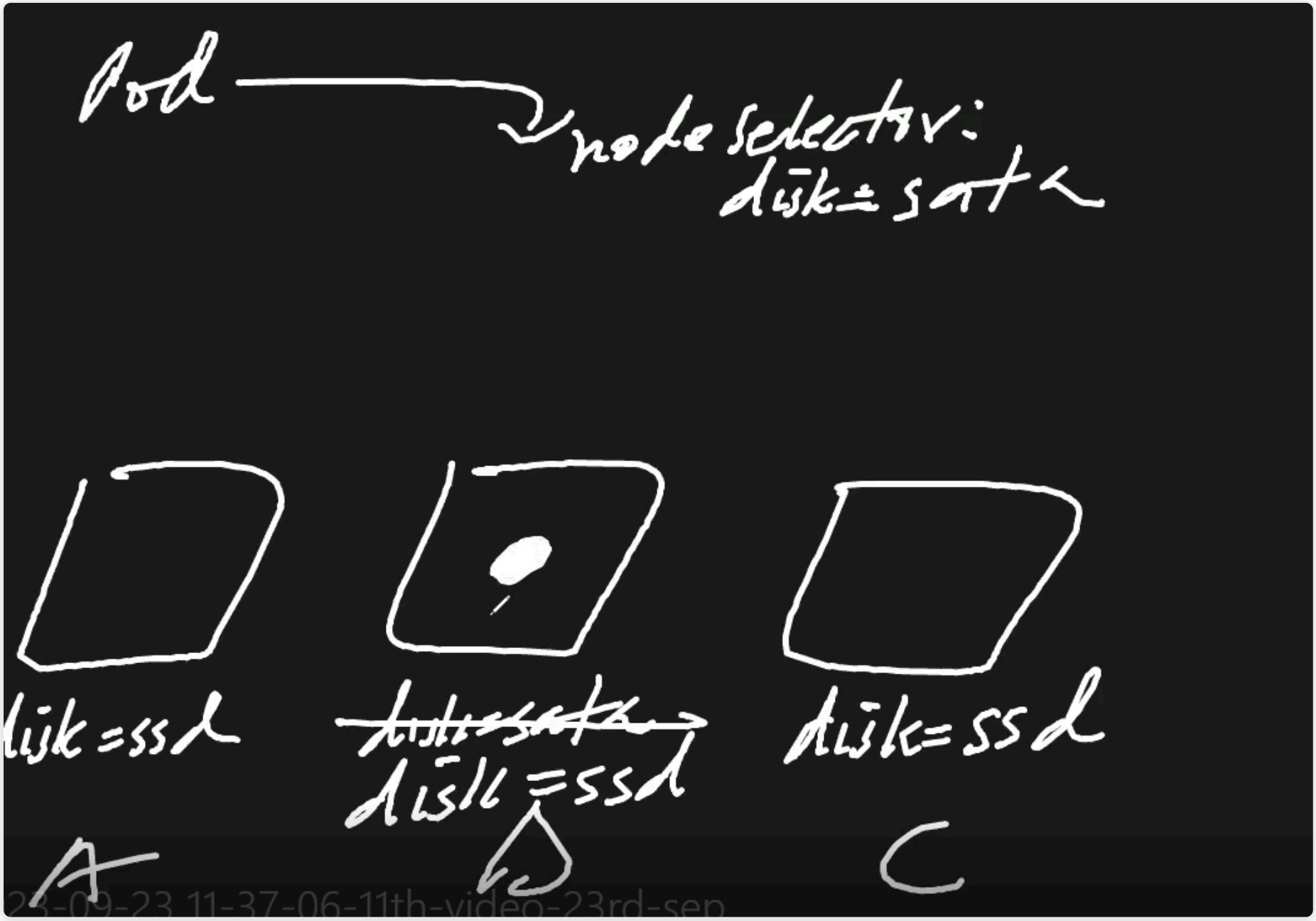
```
#
apiVersion: v1
kind: Pod
metadata:
  name: nnappone
  namespace: learning
  labels:
   app: nnappone
spec:
  nodeName: nodeone.example.com #this will assign pod to specific node
  containers:
    - name: networknuts-app
      image: lovelearnlinux/webserver:v1
      resources:
        limits:
          memory: "500Mi"
        requests:
          memory: "300Mi"
```

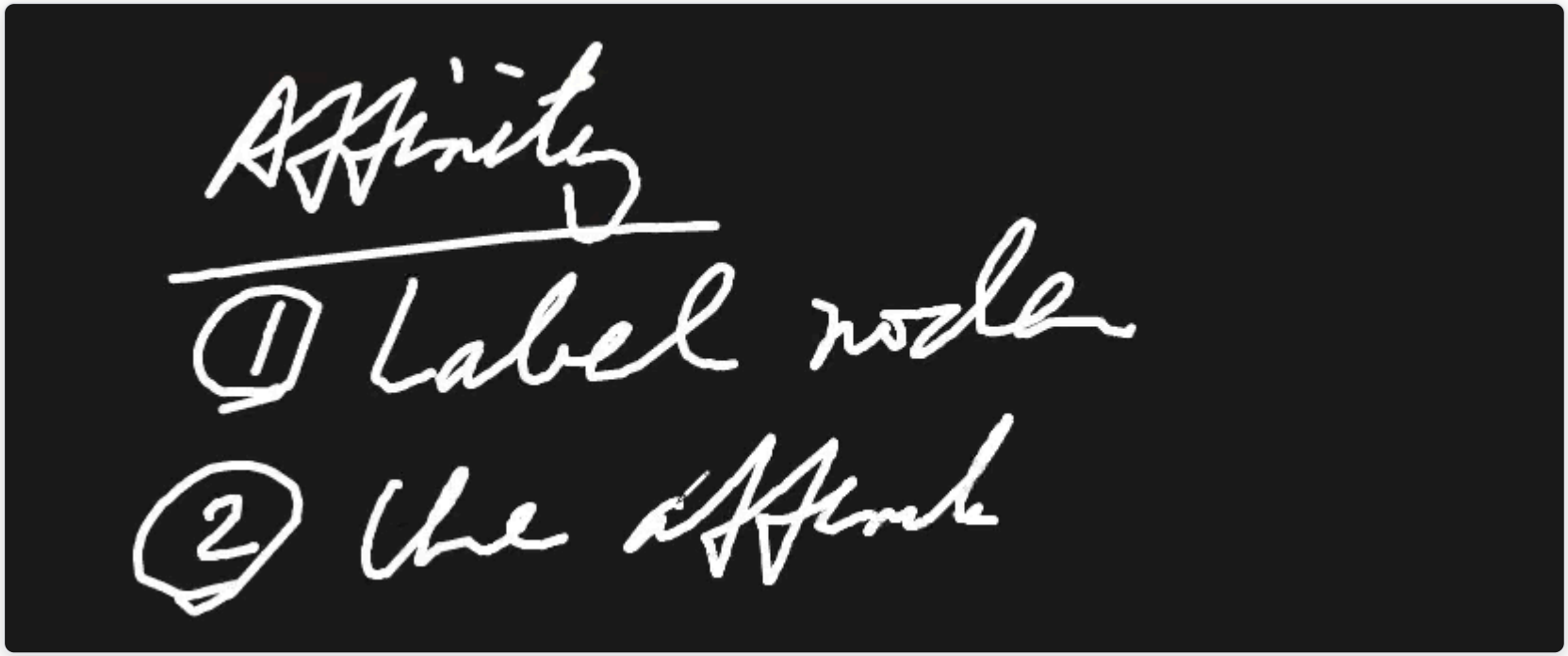
```
root@manager:~/weekends# kubectl create -f specific.yaml
pod/fix created
root@manager:~/weekends#
root@manager:~/weekends# kubectl get pods
NAME                      READY   STATUS              RESTARTS   AGE
fix                       0/1     ContainerCreating   0          3s
webapp-cbf87bdf5-5cwzp    1/1     Running             0          3m48s
webapp-cbf87bdf5-blkgg    1/1     Running             0          3m52s
```

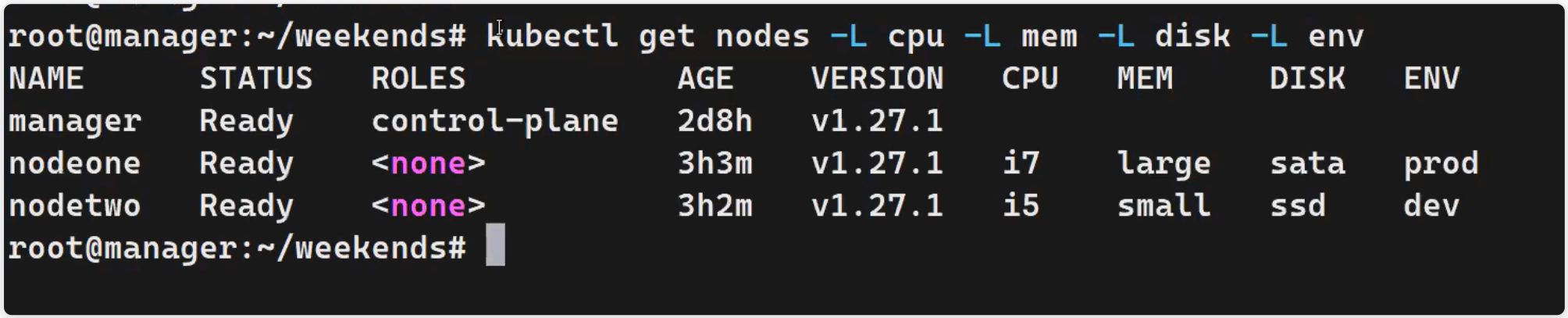
```
    DownwardAPI:               true
QoS Class:                     Burstable
Node-Selectors:                <none>
Tolerations:                   node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                               node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason   Age   From     Message
  ----    ------   ----  ----     -------
  Normal  Pulled   15s   kubelet  Container image "lovelearnlinux/webserver:v1" already present
 on machine
  Normal  Created  15s   kubelet  Created container networknuts-app
  Normal  Started  14s   kubelet  Started container networknuts-app
```

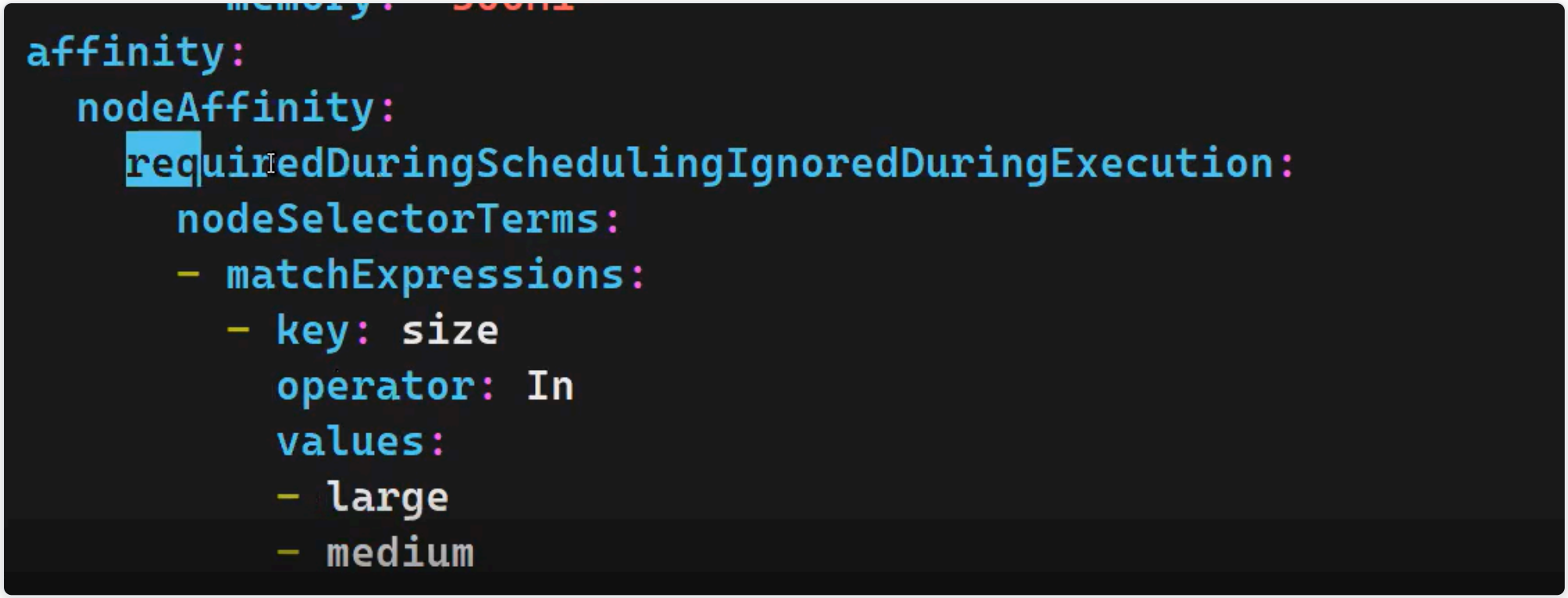

Affinity
Pod
Node
Anti Affinity
Pod
Node

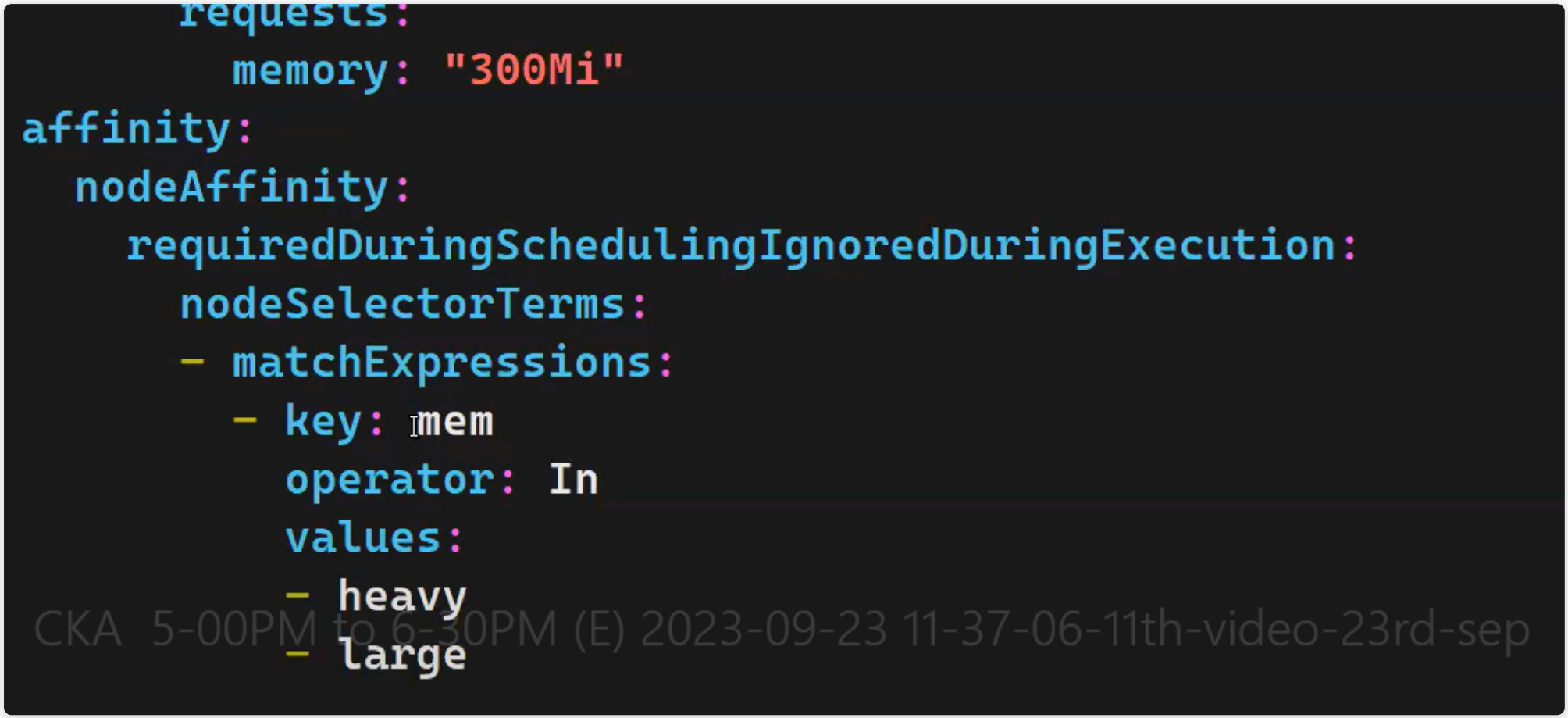

Affinity
Pod
Node
Anti Affinity
Pod
Node
AND
OR
CANNOT


Pod
node selector:
disk=ssd
disk=ssd
disk=ssd
A
C


Affinity
1 Label nozzle
2 the affinity

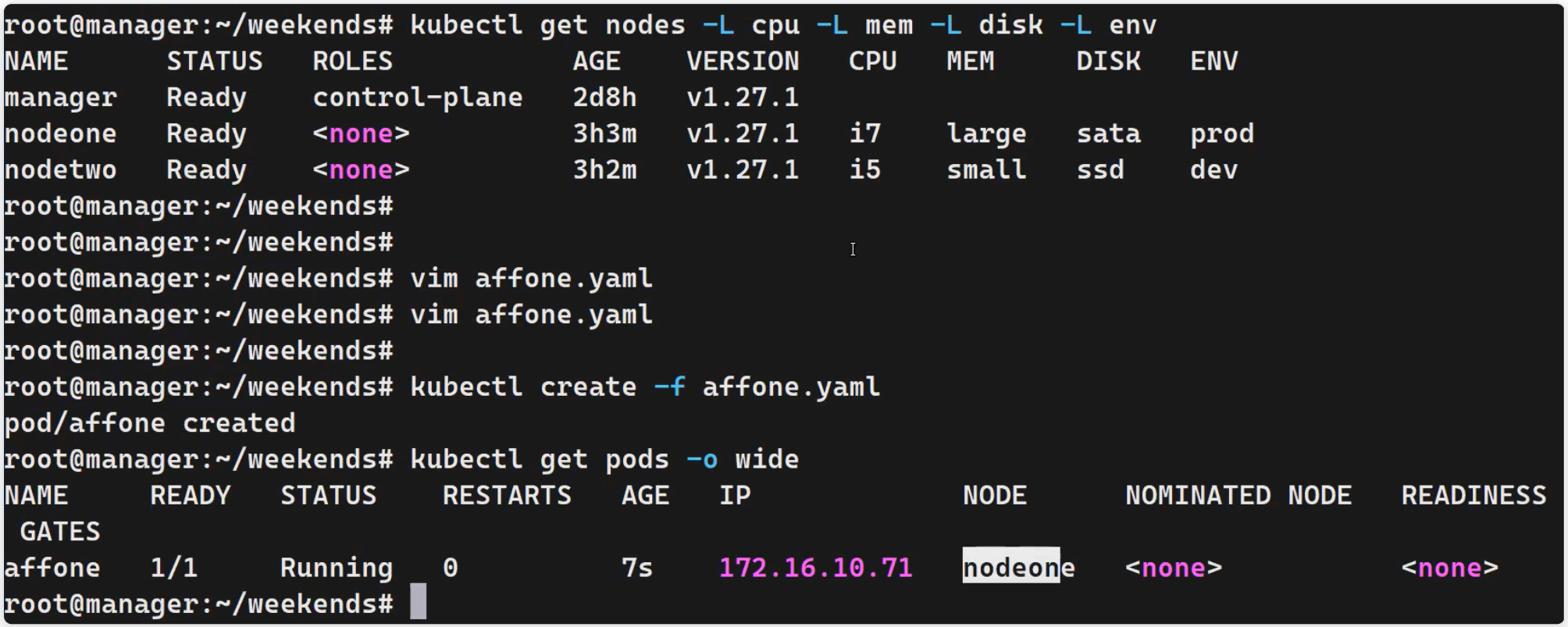

root@manager:~/weekends# kubectl get nodes -L cpu -L mem -L disk -L env
NAME STATUS ROLES AGE VERSION CPU MEM DISK ENV
manager Ready control-plane 2d8h v1.27.1
nodeone Ready <none> 3h3m v1.27.1 i7 large sata prod
nodetwo Ready <none> 3h2m v1.27.1 i5 small ssd dev
root@manager:~/weekends#


affinity:
nodeAffinity:
requiredDuringSchedulingIgnoredDuringExecution:
nodeSelectorTerms:
- matchExpressions:
- key: size
operator: In
values:
- large
- medium

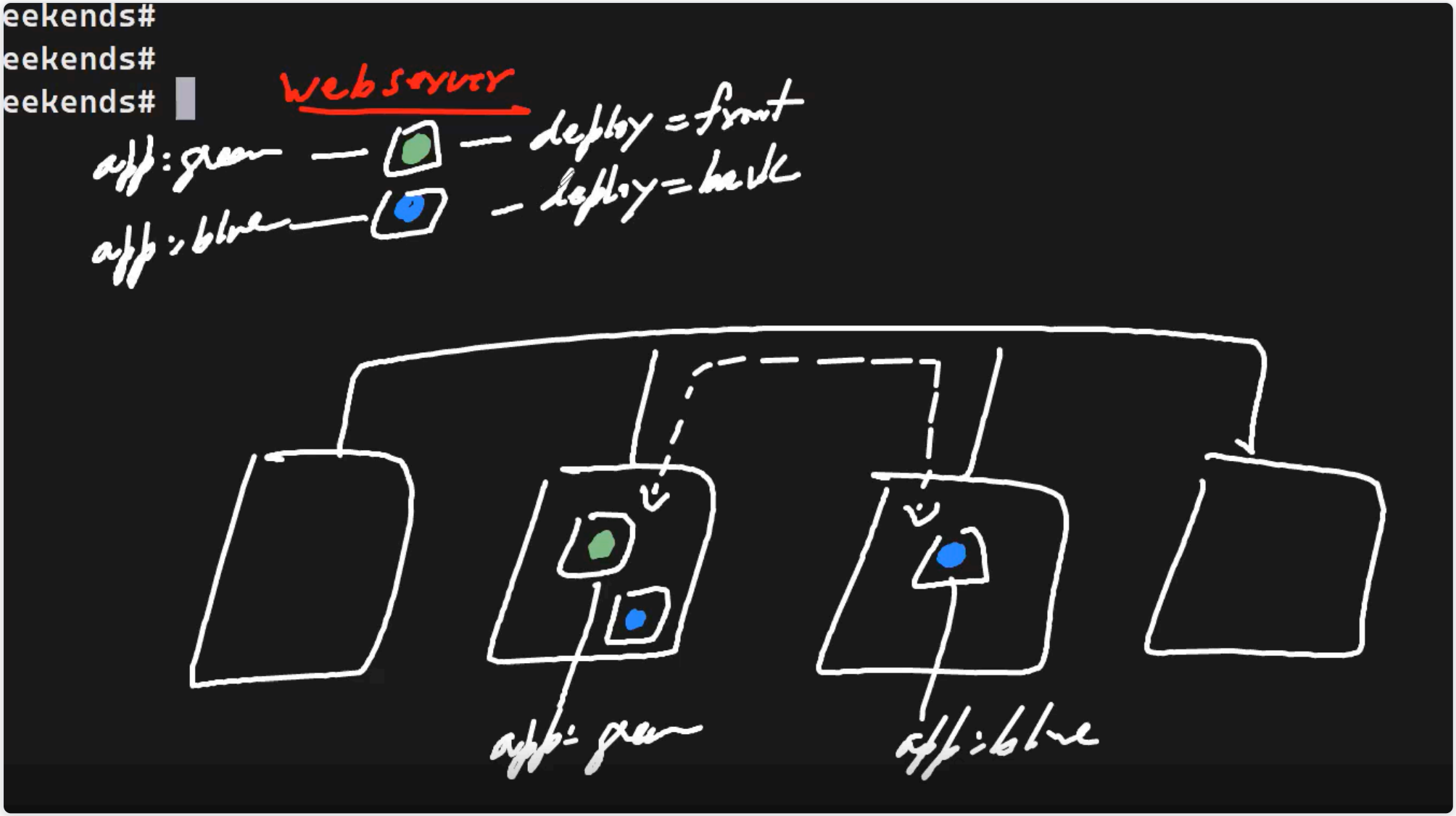


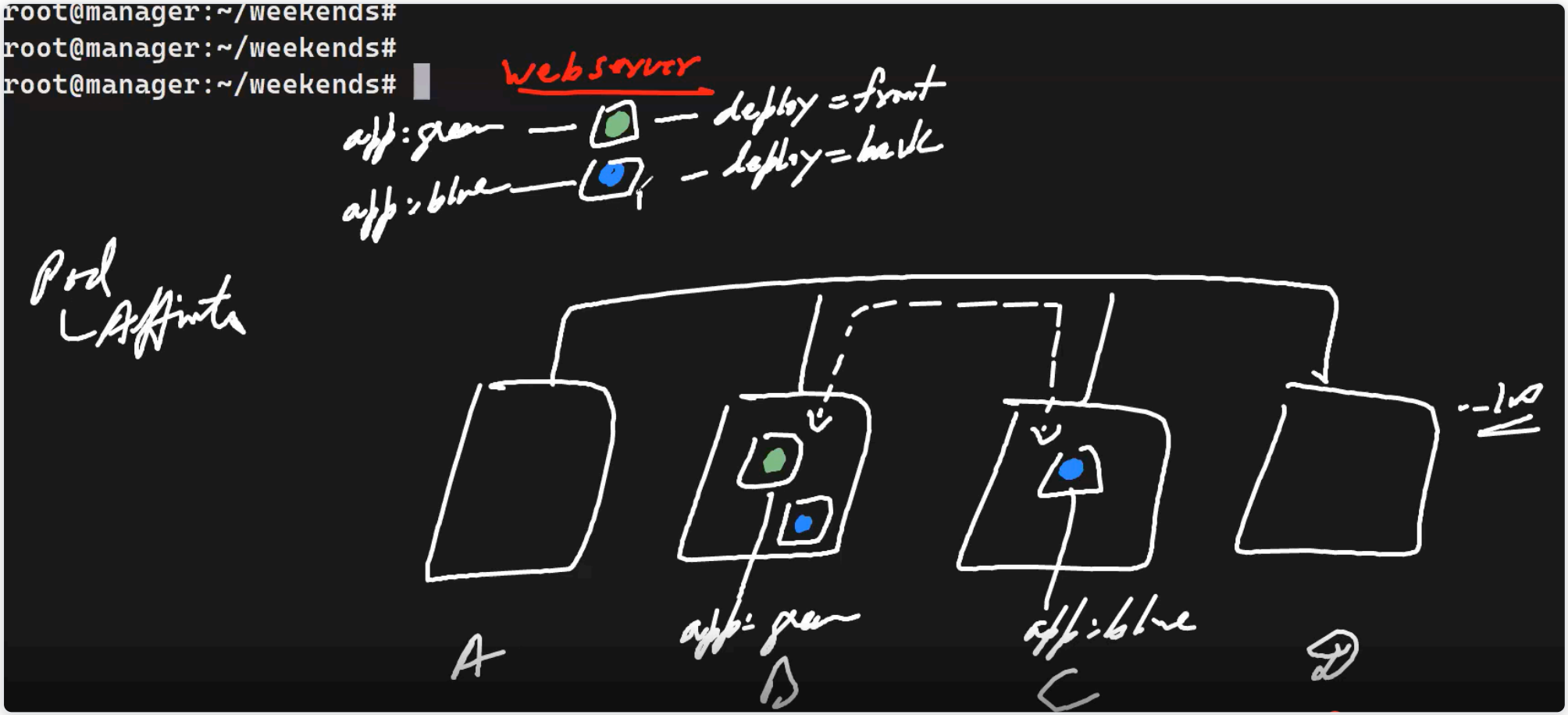
CKA 5-00PM to 6-30PM (E) 2023-09-23 11-37-06-11th-video-23rd-sep

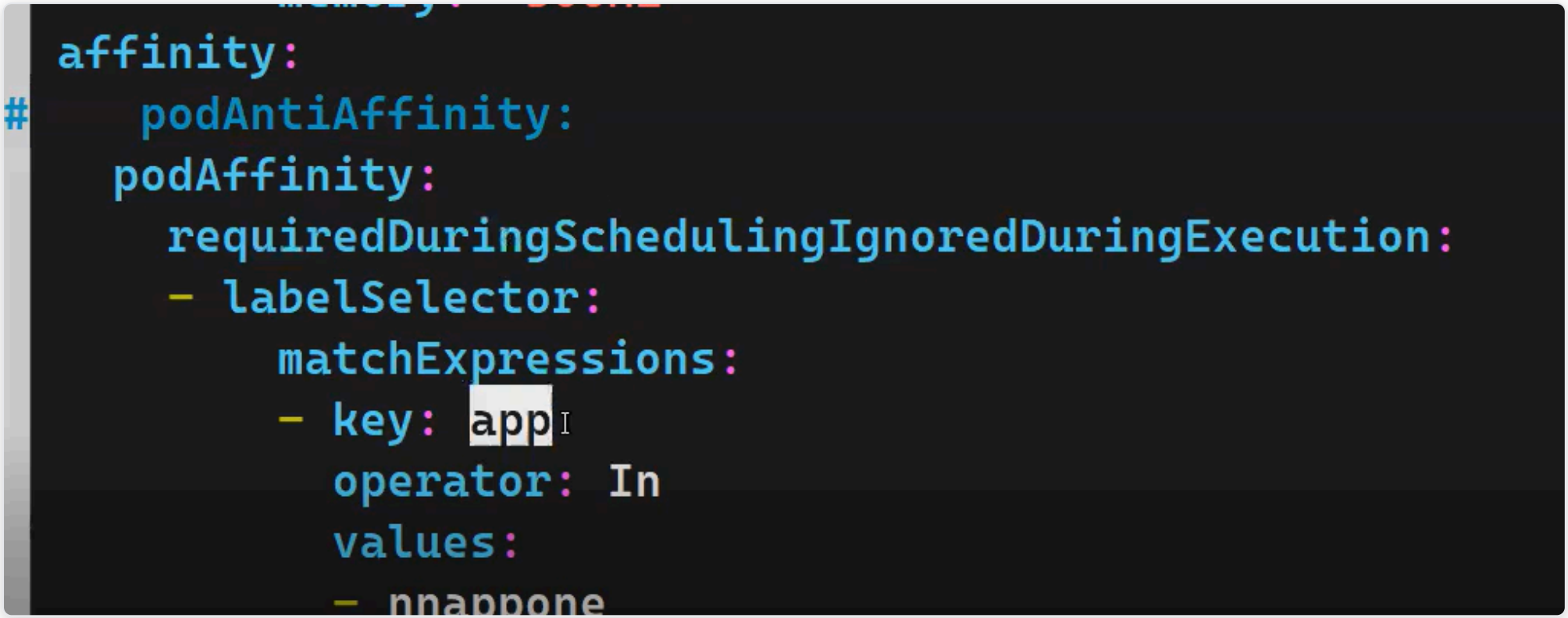
```yaml
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
      - matchExpressions:
        - key: mem
          operator: In
          values:
          - heavy
          - large
```


root@manager:~/weekends# kubectl get nodes -L cpu -L mem -L disk -L env
NAME STATUS ROLES AGE VERSION CPU MEM DISK ENV
manager Ready control-plane 2d8h v1.27.1
nodeone Ready <none> 3h3m v1.27.1 i7 large sata prod
nodetwo Ready <none> 3h2m v1.27.1 i5 small ssd dev
root@manager:~/weekends#
root@manager:~/weekends#
root@manager:~/weekends# vim affone.yaml
root@manager:~/weekends# vim affone.yaml
root@manager:~/weekends#
root@manager:~/weekends# kubectl create -f affone.yaml
pod/affone created
root@manager:~/weekends# kubectl get pods -o wide
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES
affone 1/1 Running 0 7s 172.16.10.71 nodeone <none> <none>
root@manager:~/weekends#


requiredDuringSchedulingIgnoredDuringExecution:


eekends#
eekends#
eekends#
webserver
app:green
app:blue
deploy=front
deploy=back
app:green
app:blue


root@manager:~/weekends#
root@manager:~/weekends#
root@manager:~/weekends#
Webserver
app: green
deploy = front
app: blue
deploy = back
Pod Affinity
A
app: green
app: blue
--no


affinity:
podAntiAffinity:
  podAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector:
        matchExpressions:
        - key: app
          operator: In
          values:
          - nnappone

```yaml
affinity:
    podAntiAffinity:
  podAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector:
        matchExpressions:
        - key: app
          operator: In
          values:
          - nnappone
      topologyKey: "kubernetes.io/hostname"
```

```
root@manager:~/weekends# kubectl get pods -o wide --show-labels
NAME     READY   STATUS    RESTARTS   AGE     IP              NODE      NOMINATED NODE   READINE
SS GATES   LABELS
affone   1/1     Running   0          8m42s   172.16.10.71    nodeone   <none>           <none>
           app=nnappone
root@manager:~/weekends# vim testpod.yaml
root@manager:~/weekends# vim testpod.yaml
root@manager:~/weekends#
root@manager:~/weekends# kubectl create -f testpod.yaml
pod/testpod created
root@manager:~/weekends# kubectl get pods -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP              NODE      NOMINATED NODE   READINES
S GATES
affone    1/1     Running   0          10m   172.16.10.71    nodeone   <none>           <none>
testpod   1/1     Running   0          6s    172.16.10.73    nodeone   <none>           <none>
root@manager:~/weekends#
```